say you have a firewall. A plain packet filter is a firewall just as much as one of those umpteen-integrated-application-proxy, six-figure-price-tag devices. Remember this the next time someone says he has a firewall.

Realistically, a firewall is not a security device. It is a point of policy enforcement.[2] The firewall doesn't secure anything; it prevents access to certain services. But blocking access doesn't secure inherently insecure services—it just means you can't access them. If your firewall permits access to a service, the firewall doesn't add any security to that service.

In order to build an effective firewall, you must understand TCP/IP. If Chapter 11 was a revelation to you, get a copy of *The TCP/IP Guide* (No Starch Press, 2005). Read it. Mark it up. Highlight it. And read it again.

Many of the examples in this chapter assume that you are building a firewall. This means that your host has two or more network interfaces (including VLAN interfaces) and that you want to protect the network on one side from the network on the other side. While this is a popular application for OpenBSD, everything covered here works just as well on individual hosts. I filter packets on lone web servers, on desktops, and on any host sitting naked on the Internet.

Enabling and Configuring PF

OpenBSD enables PF by default at system boot with these *rc.conf* variables:

```
pf=YES
pf_rules=/etc/pf.conf
```

To disable PF at boot, set `pf` to `NO` in *rc.conf.local*.

The default configuration file for PF is */etc/pf.conf*. There's nothing special about this file—it's just a standard location. The `pf(4)` kernel interface doesn't read the file directly; the PF control program `pfctl(8)` reads the file and sends the configuration to the kernel.

The default PF configuration (hard-coded in */etc/rc*) blocks all network traffic except for ICMP and SSH. During boot, PF replaces those defaults with rules from */etc/pf.conf*. If an error in *pf.conf* renders the file unparsable when the system boots, PF can't load those rules; instead, it retains the default configuration. You'll be able to connect to your machine to correct your rules, but that's about it. (And, as anyone who administers remote firewalls can tell you, this ability can save you a lot of driving and phone calls.)

Running PF by default, even with a permissive ruleset, cleans up incoming traffic before the rest of the kernel has to deal with it. PF reassembles packets before handing them to the kernel, and obviously bogus traffic, such as packets too short to be legitimate, is discarded.

2. Blatantly stolen from Henning Brauer. Thankfully, he's so sick of this book by now, he won't notice.

If you want to forward packets between interfaces (that is, act like a "firewall"), tell the kernel to forward packets with the `net.inet.ip.forwarding` and `net.inet6.ip6.forwarding` sysctls. (See */etc/sysctl.conf* for commented-out examples.)

```
#net.inet.ip.forwarding=1
#net.inet6.ip6.forwarding=1
```

Remove the pound signs and reboot, or use `sysctl(8)` to enable and disable packet forwarding on the fly.

Packet-Filtering Basics

Packet filtering is comparing packets to a list of rules and accepting, rejecting, or altering them as those rules dictate. As a network administrator, you get to decide which packets are naughty and which are nice. When you filter packets for a single host, you can legitimately call that host *hardened*. (The word *hardened* means almost exactly what *firewall* means: nothing.) When you send all packets on your network through a single host that filters packets, you have a basic firewall.

A basic packet filter might allow you to filter based on only the TCP or UDP protocol number. Some don't even allow you to filter by ICMP type or cannot cope with protocols other than those enumerated in the GUI. PF, however, can cope with almost anything you throw at it. If you need a machine to communicate with another over IP protocol 184, PF will support you. Many commercial firewalls won't let you pass such traffic, or claim that they do but throw a tantrum if you actually try it.

Packet-Filtering Concepts

Chapter 11 described how TCP connections can be in a variety of states. A TCP connection that is just starting goes through a three-way handshake process. A client requests a connection by sending a synchronization request, or SYN, packet to the server. The server responds by sending the client an acknowledgment of the SYN, as well as its own SYN request, or a SYN+ACK packet. The client responds with its own ACK.

Every part of this three-way handshake must complete for any actual data to transfer between the two machines. Your packet-filtering rules must permit each part of the three-way handshake and the subsequent data transmission. PF automatically recognizes these three-way handshakes and tracks them through *stateful inspection*.

Stateful Inspection

PF maintains a list of permitted connections that have completed connection setup, which is called a *state table*. When a client sends out a SYN packet, PF records that packet in a table and waits for a corresponding SYN+ACK packet. If a SYN+ACK packet arrives at PF, but PF has no record of a corresponding SYN request, the SYN+ACK packet is rejected.

PF has a series of built-in timeouts that dictate how long idle connections remain in the state table, how long to wait for each stage of the three-way handshake, and so on. The state table is self-maintaining, and I've never had to adjust any of these timeouts. (On occasion, I have needed to increase the maximum size of the state table.)

UDP is technically stateless, but some applications expect a certain amount of state. When your system transmits a UDP packet, the application might well expect a UDP packet or 10 in response, or no packets, depending on the application.

DNS queries are a common example of UDP packets flowing back and forth, and while UDP has no state, DNS certainly does. (ICMP behaves similarly.) You can have PF either expect this back-and-forth or not, by adding these flows to the state table as your protocol dictates.

NOTE *PF can also operate without stateful inspection, allowing traffic to and from hosts and ports based on individual packet characteristics.* Stateless filtering *is slower than stateful inspection, harder to correctly configure, and generally considered less secure and less useful than stateful inspection.*

Packet Reassembly

Packets can be mangled during transit, usually by *fragmentation*. Part of a packet filter's job is to sensibly *reassemble* those packets. PF can reassemble and rationalize packets in a variety of ways. (Old versions of PF called this *scrubbing*.)

Default Accept vs. Default Deny

One of the essential concepts in packet filtering is the question of default accept versus default deny:

- A *default accept* stance means that you allow any type of connection except what you specifically deny. The default PF rules are an example of a default accept stance.
- A *default deny* stance means that you allow only explicitly permitted connections. All other connections are refused.

Once you have chosen your default, you can adjust your rules to hide or reveal network services as needed. In today's world, I recommend default deny on all systems, because this stance protects new services as they are added to a system. In most environments where I've seen a default accept stance used in the past decade, it's because the system administrators did not understand the network protocol they were using. This is particularly common in VoIP installations (yes, you *can* packet filter VoIP servers!)

In addition to packet filtering and reassembly, PF offers several other important features, including NAT, connection redirection, and bandwidth management, to name a few. We'll consider each separately. All are configured in *pf.conf* and managed with `pfctl(8)`.

"My Network Can Do No Wrong"

Many network administrators who build a firewall carefully filter and restrict incoming traffic, but only apply minimal restrictions on outgoing traffic. While control of incoming traffic is among the most in-your-face issues of network management, control of outgoing traffic is also important.

Even if you trust your users, malware can convert a skilled engineer's workstation into a garbage-spewing pest. Do not assume that your network can do no wrong. It can be malicious, and one day it will be, but careful traffic control can minimize the damage you inflict on your neighbors, clients, customers, and reputation.

Is there any reason for your staff desktops to connect to any random remote mail server? If not, block it, and even if a workstation is infected with a spambot, the rest of the world won't blacklist you. Is there any reason for your users to connect to remote DNS servers, or should they use your company's? Block outbound DNS, and prevent your users from becoming unwitting amplifiers of denial-of-service attacks. I strongly recommend a default deny stance for outbound as well as inbound traffic, and explicitly allowing desirable traffic.

Some networks might be exceptions, of course. If every system on your network runs OpenBSD, you're pretty safe from routine malware, but already we see malware targeting televisions, Blu-ray, streaming media players, and other appliances with network connectivity. Protect yourself now.

Anytime that you catch yourself thinking that your network can do no wrong, stop and remind yourself that you are not as smart as the combination of every malware author in the world.

What Packet Filtering Doesn't Do

Packet filtering controls network connections based entirely on TCP/IP protocols and related characteristics, such as port numbers. If you want to block all traffic from certain IP addresses, packet filtering is your friend. If you want to allow only connections to a particular TCP/IP port, packet filtering will work for you. If you want to allow entrance only to packets with the ECN flag set, but no other flags, PF will support you (even though that's a pretty daft thing to do).

You can filter protocols that operate at a logical protocol layer such as IPsec, SKIP, VINES, and so on, but only on the network protocol. If it's a different protocol layer, PF can't help.

NOTE *PF can even filter by MAC address. There's special support for this specific media layer protocol via tags added on* `bridge(4)` *interfaces, as documented in* `ifconfig(4)`.

Similarly, PF doesn't know anything about applications or application protocols. If you allow TCP/IP connections to port 25 on a server within your network, you might think that you're allowing connections to the mail server on that host. Actually, you're allowing connections to whatever daemon happens to be running on port 25 on that host! PF doesn't recognize

an SMTP data stream; it sees only that the connection goes to port 25. (I have a system that offers SSH on many ports commonly assigned to other services, just so I can saunter past whatever naïve packet filter I happen to be stuck behind.)

PF Components

Before we dive into PF, let's look at the basic components of packet filtering on OpenBSD. In addition to the `pf(4)` kernel module, we'll look at the packet filter control program and the configuration file */etc/pf.conf*. Knowledge of interface groups also helps.

Packet Filter Control and Configuration

Use the packet filter control program `pfctl(8)` to manage, configure, and extract information from PF. You can see the current packet filter rules and settings, connections being processed, the state of the TCP/IP transactions, debugging information, and all kinds of other details. You can also parse rules files and install them in the actual packet filter.

You'll see many different options for `pfctl`, addressing every aspect of packet-filter management. Many of these are rather lengthy, but you need to type only as much of the word arguments to make a command unique. For example, instead of typing `pfctl -s rules`, you can get away with `pfctl -sr` because no other argument to `pfctl -s` begins with an `r`. That said, I give all examples in their full form, as it's impossible to guarantee that OpenBSD won't add some other argument that begins with `r` in the future.

I focus on using `pfctl` for viewing PF output, but OpenBSD also includes PF views in `systat(1)`. For a dynamic display of PF activity, somewhat like `top(1)` for the network, look at `systat`. Run `systat` by giving the name of the view as an argument, such as `systat pf`. And, as always, any time you want more detail from `pfctl`, add one or two `-v` arguments for verbose mode.

You configure PF in */etc/pf.conf*. The *pf.conf* file contains statements and rules, whose format varies with the features they configure. You'll be very good friends with this file before we're through.

Interface Groups

OpenBSD lets you put interfaces in named groups, which you can refer to in PF rules. This abstracts away the actual physical interface, and lets you build policy-based rulesets. Take a look at this interface:

```
# ifconfig em0
em0: flags=8843<UP,BROADCAST,RUNNING,SIMPLEX,MULTICAST> mtu 1500
...
        groups: egress
...
```

This interface is in the `egress` group. An interface is assigned to the `egress` group if a default route is reached over it.

To move this interface to a new group, `dmz`, remove it from the `egress` group and add it to the `dmz` group. An interface group is created when you assign the first interface to it, and one interface can be in any number of groups.

```
# ifconfig em0 -group egress
# ifconfig em0 group dmz
# ifconfig em0
em0: flags=8843<UP,BROADCAST,RUNNING,SIMPLEX,MULTICAST> mtu 1500
...
        groups: dmz
...
```

You can now write rules that reference interface groups instead of specific interfaces.

PF Configuration

Let's dismantle the default *pf.conf* from an OpenBSD system and identify some parts. Many of the default entries are commented out, but identifying them will help you understand how the components fit together.

It begins with an *option*:

```
set skip on lo
```

Options turn features on and off, or set general rules on how other features behave. The `skip` option disables PF on a per-interface basis.

Next comes the `anchor` setting:

```
anchor "ftp-proxy/*",
```

An *anchor* is a set of dynamic sub-rules for packet filtering. If a packet hits an anchor as it's processed through the filter rules, it's dropped into this sub-ruleset for further processing. `pfctl` can change the rules running in the kernel, and an anchor is a way of saying, "Add new rules here."

Anchors are generally used for letting outside software add rules to the firewall. For example, FTP is a complicated protocol that requires all sorts of firewall rules. OpenBSD includes an FTP proxy that dynamically adds the necessary rules for permitted FTP connections.

Then come two *packet-filtering* rules:

```
pass in quick inet proto tcp to port ftp divert-to 127.0.0.1 port 8021
pass
```

The first is a rule to support FTP traffic, in combination with the FTP anchor. We'll look at anchors and FTP handling in more detail in the next chapter. The other is a much simpler packet-filtering rule, which permits all traffic.

Up next are two *tables*, which are lists of IP addresses:

```
table <spamd-white> persist
table <nospamd> persist file "/etc/mail/nospamd"
```

External programs can dynamically alter tables, and you can add addresses to tables directly within *pf.conf* or in an external file. These two tables are used by the antispam software `spamd(8)`.

After the tables is another packet-filtering rule:

```
pass in on egress proto tcp from any port smtp \
    rdr-to 127.0.0.1 port spamd
```

This rule is interesting in that it refers to an interface group. Traffic is permitted in, as long as it arrives on an interface in the `egress` group.

And the final rule is as follows:

```
block in on ! lo0 proto tcp to port 6000:6010
```

This packet-filtering rule stops traffic. If a packet arrives on any interface except the loopback interface, and the packet is a TCP protocol going to port 6000 through 6010 inclusive, it is blocked.

This is the sort of thing you'll see in *pf.conf*. Let's dive into some specifics of filtering rules.

Filtering Rules

Filtering rules are the heart of PF. You can use PF without doing any of the fancy redirection, address translation, load balancing, or redundancy, but packet filtering is the bedrock on which most of these features are based. To start with, however, basic packet filtering is defined as access control for network packets by source, destination, protocol, and protocol characteristics.

PF processes filtering rules in order. The last rule that matches a packet is acted on. A typical packet-filtering rule looks like this:

```
❶pass ❷in ❸on egress ❹proto tcp ❺from any ❻to 192.0.2.12 ❼port 80
```

The first word of the filter rule is a keyword that describes the results of this rule ❶. PF will either `pass` or `block` packets that match a rule. (There's also `match`, which we'll look at in the next chapter.) The rest of the line is a description of matching packets. If the packet matches the description, the rule is applied.

The second statement is the direction the packet is going. Packets are either going in or out. In this rule, the packet is going `in` ❷—it is entering the system. Not only do we define a direction, but we also define an interface group. Packets must be entering this system on an interface in the `egress` group to match this rule ❸.

We then have several statements that define traffic characteristics. (This rule is almost like a regular expression for TCP/IP.) This rule applies to TCP connections ❹, coming from any IP address ❺, if the connection is made to the IP address 192.0.2.12 ❻ on port 80 ❼.

If a packet matches all of these characteristics, it can pass. If any of these characteristics isn't matched, the packet does not match this rule, and PF continues processing the rules, looking for a matching one.

TCP and UDP rules implicitly check connection state. A TCP packet that matches this rule needs to be a SYN packet, the start of a standard TCP/IP connection. PF uses the state table to manage follow-up packets in the same connection (see "Filtering Rules and the State Table" on page 411).

Default Permit or Default Deny

I touched earlier on the idea of default accept versus default deny. Set this stance at the beginning of your packet-filtering rules with one of the following two statements:

```
pass
block
```

The default *pf.conf* has a default pass stance, but it's for people who haven't yet configured a firewall. I recommend starting your filter rules with a lone `block` statement, and then adding rules to explicitly permit desirable traffic. Remember that the last matching rule wins.

Packet Pattern Matching

One of the most intensive parts of PF is the syntax used to describe packets. Most filter rules describe packets by protocol, port, direction, and other characteristics. PF compares each arriving packet to the state table, and if the packet isn't part of the state table, it compares the packet to the filter rules. If the rule matches the packet description, the packet is passed or blocked as desired.

Once you define whether you're in a default accept or default deny stance, the filter rules describe exceptions to your default. So if you block packets by default, most of your filter rules will be `pass` statements that describe particular desirable connections.

Direction

The keywords `in` and `out` describe the direction the packets are going. In many commercial firewalls, the word `in` means traffic entering the protected network, and `out` refers to traffic leaving the protected network. OpenBSD does not magically know which side of the network is protected and which is not. As far as PF knows, it's managing traffic between two interfaces. The keyword `in` means traffic flowing into the machine from the network, and `out` means traffic leaving the machine and entering the network.

When you see `in` or `out` in a PF rule, do not think about your network as a whole. Instead, imagine that you're very small and sitting on your CPU, grilling steaks over the heat sink and watching packets enter and leave the computer. You cannot see what lies beyond the case, just the packets as they come and go. Packets coming in are approaching you, and packets going out are receding.

Interface Matching

The `on` keyword describes an interface or interface group to which this rule applies. You must specify an interface.

If you want a rule to match every interface on the system, use the interface name `all`. This example stops all traffic entering the machine on the interface `fxp0`, but allows all traffic leaving the system on the interface group `egress`:

```
block in on fxp0
pass out on egress
```

This ruleset implies that interface `fxp0` is special for some reason, so it's not treated like the rest of the `egress` group.

Address Families

Rules can apply to specific address families, either `inet` for IPv4 or `inet6` for IPv6. Here's how to prohibit IPv4 but permit IPv6:

```
block in on egress inet
pass in on egress inet6
```

Presumably, you have later rules that more tightly restrict IPv6.

Network Protocol

PF can recognize almost any network protocol by number or name. The `proto` keyword tells PF to match a protocol. Network protocols can be given by name from */etc/protocols*, protocol number, or even a list (see "Using Lists" on page 413).

```
block in on egress proto tcp
pass in on egress proto udp
```

You can use this to pass protocols other than IP and IPv6. Here's how to allow the protocols necessary for IPsec:

```
pass in on egress proto esp
pass in on egress proto ah
```

This functionality somewhat overlaps the `inet` and `inet6` statements. If you prefer, you could explicitly allow IP, ICMP, TCP, UDP, and all the various IPv6 protocols.

Source and Destination Address

Almost every filter rule specifies a source and/or destination address.

```
pass in on egress from 198.51.100.0/24 to 192.0.2.0/24
```

IP addresses can appear either as individual addresses or as an address with a netmask (as shown in the preceding example). The keyword `any` means any IP address. The keyword `all` is shorthand for "from any to any."

You can also use hostnames instead of IP addresses. `pfctl` will check the IP address of the host when loading the rules, and insert the actual IP address into the rules.

```
pass in on egress from www.michaelwlucas.com
```

If the IP address of the host changes, PF won't notice until you reload the rules with `pfctl`. If the hostname cannot be found, the rules won't parse, and `pfctl` will not be able to load them. I recommend not using hostnames in filter rules, much as I recommend not wearing medieval plate armor while swimming, but it is an available option.

To say "anything but this address," use the exclamation point as a negation character.

```
block in from !192.0.2.0/24
```

This says "block everything except the addresses 192.0.2.0/24." That's not the same as saying "pass 192.0.2.0/24," but it can help simplify your rules.

You can also use lists, macros, and tables as IP addresses. Lists and macros are discussed later in this chapter, and tables are covered in the next chapter.

Source and Destination Variants

You can use the name of an interface or interface group instead of an IP address.

```
pass out on egress from egress
```

This lets traffic leave via the `egress` interface group, from any IP address on any interface in that group, to any IP address.

If you put the interface name or group in parentheses, PF updates its rules whenever the IP address on the interface changes. This is useful for dial-up connections, or if you add and remove IP addresses from an interface.

```
pass out on egress from (egress)
```

You can specify a network that is directly attached to an interface or an interface group by following the name with `:network`.

```
pass in on egress from egress:network
```

Suppose the `egress` group has only one interface, and that interface has an IP address of 192.0.2.88/25. This rule would translate to the following:

```
pass in on egress from 192.0.2.0/25
```

This rule means that any host on the local network to an `egress` interface can communicate anywhere. When you add another interface to the `egress` group, the rules automatically update to accommodate the new interface's network.

To filter on broadcast traffic for an interface or group, use the `:broadcast` modifier.

```
block in on egress from egress:broadcast
```

Again, suppose that the `egress` group has only one interface, and that interface has an IP address of 192.0.2.88/25. This rule would translate to the following, blocking broadcast traffic on the local subnet:

```
block in on egress from 192.0.2.127
```

Use the `:peer` modifier to indicate the IP address of the far side of a point-to-point link, such as a dial-up connection.

```
pass in on egress from egress:peer
```

Here, we completely trust our dial-up provider.

Interface Main Address

To use only the first IP address on an interface, add the `:0` modifier with an interface or group name.

```
pass out on egress from (egress:0)
```

The `egress` interface group might have 98 IP addresses scattered across three interfaces, but only one address on each interface is the first address. This host can communicate out through the `egress` interface group, but only from primary IP addresses. The aliased IP addresses cannot initiate outbound connections.

The problem with the `:0` modifier is that the kernel has a very weak idea of what is the "first" address on an interface. The kernel has a list of addresses associated with an interface. The address at the top of this list is the "first" or "main" address at the moment, but this address can change. If this might cause problems, specify an IP address in your rule rather than rely on `:0`.

You can attach `:0` to any of the other interface modifiers, that is, to IP addresses other than the first from the rule. OpenBSD can't tell if IP addresses on remote machines are aliases or actual IP addresses, but you can prohibit traffic to or from aliases on the local machine.

Note that the first address on an interface is either an IPv4 address *or* an IPv6 address. If you want to allow the first address of each protocol, specify the address family in the rule.

```
pass out on egress inet from egress:0
pass out on egress inet6 from egress:0
```

Otherwise, PF will use only the first address it sees, regardless of address family.

Source and Destination Port

Filter rules can describe TCP and UDP ports.

```
pass in on egress proto tcp from any to 192.0.2.12 port 80
```

This example permits access to TCP port 80 on the server 192.0.2.12. Presumably, this is a web server.

You could use a service name from */etc/services* instead of a port number, or even use a list (as described later in this chapter). You can also use ranges, as shown in Table 21-1.

Table 21-1: Port Ranges

Symbol	Meaning
!=	Not equal
<	Less than
>	Greater than
<=	Less than or equal to
>=	Greater than or equal to
><	Range
<>	Inverse range

For example, to specify all ports over 1024, you could use the greater-than operator (>).

```
pass in on egress proto tcp from any to 192.0.2.12 port > 1024
```

To specify all ports between 1000 and 2000, excluding both 1000 and 2000, use the range operator (><).

```
pass in on egress proto tcp from any to 192.0.2.12 port 1000 >< 2000
```

To include ports 1000 and 2000 in your range, use the inclusive range operator (`:`). (Note that you cannot have space on either side of the colon.)

```
pass in on egress proto tcp from any to 192.0.2.12 port 1000:2000
```

To pass traffic on all ports less than 1000 and greater than 2000, use the inverse range operator (`<>`).

```
pass in on egress proto tcp from any to 192.0.2.12 port 1000 <> 2000
```

Ranges let you express large numbers of ports in very few rules.

A Complete Ruleset

The following is a complete ruleset for a desktop machine, using many of the features described previously. We'll look at some more complicated rulesets later, but this illustrates many basic principles of PF rules.

Interface group `egress` is attached to the public network, and interface group `inside` is connected to my private network.

```
❶ set skip on lo
❷ block
❸ pass in on egress from egress:network
❹ pass in on inside from inside:network
❺ pass in on egress proto tcp from any to egress:0 port 22
❻ pass out all
```

The first rule disables packet filtering on the loopback interface ❶, and the second defines a default deny stance ❷. The second and third rules permit all connections from IP addresses directly connected to the external ❸ and internal interfaces ❹. If I install a web server on my desktop, I want to be able to view it from any machine on the network I control. Then I permit inbound SSH connections from anywhere in the world to the primary IP address on any `egress` interface ❺. Finally, I permit all outbound traffic, so my desktop can freely access the outside world ❻.

I've said before that PF rules are processed in order, and these rules illustrate that. I establish a default, blocking all traffic, and then use individual rules to carve out exceptions to that global block.

Activating Rules

For your PF rules to take effect, you must load them into the kernel using `pfctl -f`.

```
# pfctl -f /etc/pf.conf
```

First, `pfctl` reads and parses the rules file. If the file parses correctly, `pfctl` expands any variables in the file, performs any necessary DNS lookups to transform hostnames into IP addresses, and feeds the complete rules into the kernel. The kernel reads the new rules, and then swaps between

the old and new rules in one operation. At no time are the packet-filtering rules missing, scrambled, or a hybrid of the two rulesets. Also note that `pfctl -f` won't enable PF if it's disabled.

Personally, I like to know that my edited packet-filter configuration parses before the scheduled change time. It's embarrassing to announce to your team that "the new firewall configuration will be active at noon" and spend the whole time tracking down a misplaced comma or a parenthesis where you should have put in a curly brace. To test your syntax without installing the rules, use the `-n` flag with `-f`. Add `-v` for verbose mode, to see how `pfctl` expands your macros, groups, and so on.

```
# pfctl -nvf /etc/pf.conf
```

The rules might still have errors, but only errors of comprehension rather than syntax.

Loading new rules doesn't remove any existing open connections or state entries. If my old ruleset allowed outbound SSH connections, and I remove that permission from the newly installed rules, existing SSH connections remain open. I can either specifically kill those connections with `pfctl -k` or flush the state table.

Viewing Active Rules

To see how these rules are interpreted inside PF, view the currently installed ruleset with `pfctl -s rules`. Here are the rules generated by the configuration in "A Complete Ruleset" on page 409:

```
# pfctl -s rules
❶ block drop all
❷ pass in on egress inet6 from 2001:db8:4::/64 flags S/SA
❸ pass in on egress inet from 192.0.2.0/28 flags S/SA
❹ pass in on inside inet from 192.168.1.0/24 flags S/SA
❺ pass in on egress inet proto tcp to 192.0.2.5 port = 22 flags S/SA
❻ pass out all flags S/SA
```

The first rule establishes a default deny stance ❶. I then specifically allow connections from hosts on the networks local to interfaces in the `egress` group, for both IPv6 ❷ and IPv4 ❸. This desktop also accepts connections from my private network ❹.

The private network permits connections only from IPv4 addresses because the interface in the private group has only an IPv4 address. (I really should add an IPv6 address, but it hasn't caused me any trouble, so I'll probably forget all about it once again.) Then there's a rule permitting inbound SSH traffic ❺, followed by a final rule to pass all outbound traffic ❻.

If I change any IP address on my desktop, my firewall rules update to accommodate them. That's a really nice feature of interface groups. If I moved my desktop regularly, I would put the interface group names in parentheses so PF would watch for IP address changes.

NOTE *One thing you'll probably notice is that the pass rules end with* `flags S/SA`*. This means that out of the SYN and ACK flags, matching packets can have only the SYN flag set, indicating that these are requests to establish a connection. You can filter on TCP flags, but doing so requires in-depth understanding of TCP, and most people should never do it. To see how SYN and SYN+ACK packets affect connections, you need to understand the state table.*

To see how often a packet triggers each rule, add `-v` to the `pfctl` command.

To see how the rules impact traffic in a constantly updating display, run `systat rules`.

Filtering Rules and the State Table

OpenBSD tracks approved connections in the state table. Packets that are part of an approved connection are allowed to pass. Consider this rule from an earlier example:

```
pass in on egress proto tcp from any to 192.0.2.12 port 80
```

If a packet matches this rule, and it has the TCP/IP flags that indicate this is the start of a TCP connection, PF permits the connection. PF also makes an entry in the state table. If a packet arrives that matches the state table, PF passes the packet without consulting the rules.

TCP States

First, we'll look at a state table entry for a TCP connection. To view the state table, enter **`pfctl -s states`**.

```
# pfctl -s states
❶all ❷tcp ❸192.0.2.12:80 <- ❹198.51.100.227:55635 ❺ESTABLISHED:ESTABLISHED
...
```

This state table entry represents one specific connection that the packet filter approved. This state applies to all interfaces ❶. If a state applies to only one interface, you'll see the interface name here.

This TCP connection ❷ was bound for 192.0.2.12 port 80 ❸, and came from the host 198.51.100.227 port 55635 ❹. When the first SYN packet arrived from 198.51.100.227 port 55635, PF added this entry to the state table. When 192.0.2.12 sent a SYN+ACK packet back to 198.51.100.227 port 55635, PF consulted the state table. This was clearly a match to the permitted SYN packet, so PF permitted that packet, even though no explicit rule in *pf.conf* permitted that connection. Data exchange between these two hosts and these two ports proceeded.

PF knows what an actual TCP/IP data exchange looks like. There's a three-way handshake in the beginning, and a similar dance when the connection is finished and PF tracks the state of the connection. This particular connection is established on both sides ❺, meaning that the initial setup negotiation succeeded, and data can flow back and forth freely.

If your server is busy enough, and you keep refreshing the state table view, you'll catch connections in other states. Here's the same connection as the data exchange ends and is being torn down:

```
all tcp 192.0.2.12:80 <- 198.51.100.227:55635  FIN_WAIT_2:FIN_WAIT_2
```

NOTE *One possible problem with viewing the state table is that* `pfctl` *displays a snapshot. By the time your eyes scroll down the screen, the table has changed. Personally, I find that's the only way I can cope with the information. If you need to view states in a constantly updating display, in near real time, run* `systat states`.

The state table is very specific. A state table entry permitting 198.51.100.227 port 55635 to 192.0.2.12 port 80 does not permit traffic between other hosts and ports. PF knows how traffic should flow, and it won't allow traffic that isn't obviously part of an existing TCP/IP exchange. If a packet arrives from 198.51.100.227 that looks like it's part of this data exchange, except that it comes from port 55634 instead of 55635, the state table entry won't match. Similarly, if PF knows that the connection is in a `FIN_WAIT_2` state, or almost finished, a subsequent data packet with an ACK flag set won't match and will be discarded. This is because a SYN request from the same host, from the same port, should not arrive—the client should know that the port is busy closing the previous connection. A new connection should come from a different port on the client and create a new state table entry.

Without stateful inspection, you would need to write firewall rules that not only permitted incoming traffic, but also permitted the responses. Your firewall rules would need to permit outbound connections to thousands of high-numbered ports, instead of just the single ports attached to desirable connections. Filtering based on TCP flags would be nearly impossible.

NOTE *As a consultant in the 1990s, I made a couple of rent payments dismantling such rules that had been shoehorned into stateless packet filters because they just aren't realistic without stateful inspection. Plus, carefully tracking data exchanges not only simplifies rules, but also prevents a whole slew of TCP/IP-based attacks. You don't hear much about these attacks anymore, thanks to stateful inspection.*

UDP States

The state entries for UDP connections are similar to those for TCP connections.

```
all udp 192.0.2.12:53 <- 198.51.100.227:38469       SINGLE:MULTIPLE
```

This is a DNS query, bound for 192.0.2.12 port 53 from 198.51.100.227 port 38469. The client sent a single packet, and the destination replied with multiple packets. While stateful inspection cannot identify the state of this connection by flags, it can track the source and destination addresses and ports. You would need to write only a single rule permitting access to 192.0.2.12 port 53, and stateful inspection would permit the matching reply packets.

ICMP States

ICMP falls somewhere in between TCP and UDP. PF is aware of ICMP types and knows legitimate responses to ICMP packets, and by using stateful inspection, you get all of these benefits automatically. Much as you could write rules that permit specific TCP flags, you can write rules that permit certain ICMP types and codes. Most of us cannot manage that, and those of us who can know better. (ICMP errors referring to an existing TCP or UDP state are matched to the state, and don't need to be allowed separately.)

NOTE *OpenBSD's stateful inspection actually tracks more detail than source and destination addresses and ports. Add* `-v` *to the* `pfctl` *command to see more information, including timing, the number of packets passed as a result of the state, and more.*

Packet Filtering with Lists and Macros

PF includes many ways to have one rule reference several similar items, or symbolically represent something with a variable. The basic ways are lists and macros.

Using Lists

A *list* is a way to represent several similar items in one rule. You might want to use a list if, for example, you want a particular group of TCP ports open on a certain group of hosts, and your rule entries would be repetitions of one rule with minor changes. Opening ports 80 and 443 to one host requires two rules: one for each port. If you have 30 web servers, you would need 60 rules. This is a pain to maintain and error-prone, but lists let you express these common elements more easily.

A list is represented in curly braces within a rule. To make the rule more readable, you can put a comma between items.

```
pass in on egress proto tcp from any to 192.0.2.12 port {80, 443}
```

This one *pf.conf* statement creates two rules, opening both TCP ports 80 and 443 to the target host.

```
pass in on egress from any to 192.0.2.12 port = 80 flags S/SA
pass in on egress from any to 192.0.2.12 port = 443 flags S/SA
```

You could also use a list to have this rule cover multiple web servers.

```
pass in on egress proto tcp from any to {192.0.2.12, 192.0.2.13} port {80, 443}
```

This expands to four rules: one for each combination of server and port.

Remember that each entry in the list creates its own rules. The list entries do not combine to create a single rule.

Using Macros

A macro is a variable that you create and define for use within PF rules. Macros keep *pf.conf* more readable, maintainable, and manageable.

Macro names must begin with a letter, but can include letters, numbers, and underscores. You cannot give a macro a name that's used elsewhere in PF, like `pass`, `block`, or `proto`. Frequent uses of macros include interface names, network addresses, and ports.

Earlier, we saw a list that included the popular web ports 80 and 443. You could make these a macro, as follows:

```
web_ports="{80, 443}"
```

Our sample rule would then become this:

```
pass in on egress proto tcp from any to 192.0.2.12 port $web_ports
```

When combined with braces, macros can simplify your *pf.conf* file. Consider the following *pf.conf* snippet:

```
webservers="{192.0.2.12, 192.0.2.13, 192.0.2.14, 192.0.2.15}"
web_ports="{80, 443}"
pass in on egress proto tcp from any to $webservers port $web_ports
```

This expands to eight rules, but requires only three easy-to-understand configuration statements. When you add a new web server, add its IP address to the list in the `webservers` macro. What's more, you might use the `webservers` macro in dozens of places throughout your rules. Changing the IP address list once is much easier and more likely to be correct than doing so in each rule.

While you probably use interface groups to represent IP addresses local to your machine, you might have other IP addresses that you need to represent. Macros are great for this, too.

```
internal_ip="10.10.0.0/16"
```

Or if you have multiple disparate blocks, you could use a list inside the macro.

```
internal_ip="{10.0.0.0/24, 10.0.5.0/24, 10.0.10.0/24}"
```

You don't see macros or lists when viewing the running PF rules with `pfctl`; instead, you see the rules that they expand to.

A Common Error: List Exclusions and Negations

Lists can be counterintuitive, and it's easy to write lists that negate other rules. For example, this seems like it should work:

```
clients = "{192.0.2.0/24, !192.0.2.128/29}"
pass in on egress from $clients
```

The idea here is that our clients have the IP addresses 192.0.2.0/24. We want to permit all of those addresses except for the small chunk in the middle, 192.0.2.128/29. That seems reasonable, right? But much like excluding commands from `sudo(8)`, this breaks. Remember that each entry in a list expands into another rule. This creates two rules.

```
pass in on egress inet from 192.0.2.0/24 flags S/SA
pass in on egress inet from ! 192.0.2.128/29 flags S/SA
```

The first rule passes in everything from the 192.0.2.0/24 subnet. That's what we wanted. The second rule, however, passes in *everything* that's not in the subnet 192.0.2.128/29, also known as "everyone in the world"—not what we were hoping to achieve.

Similarly, negating an entire list expands to negating each individual item in the list. If you need to do this sort of exclusion, use a table, as described in the next chapter.

Sanitizing Traffic

All sorts of weird traffic arrives at Internet hosts. Some of that traffic is broken garbage. Other parts tell you that someone else is running broken garbage.

PF tries to sanitize and normalize traffic before otherwise processing it. The normalizations include discarding illegal packets, packet reassembly, and packet modification.

Illegal Packets

Some of the random stuff that arrives at a host is garbage. If a packet is shorter than the IP header, it can't be a real IP packet, and if a TCP packet is too short to include a full TCP header, it can't be a real packet.

If the packet length doesn't match the length given in the header, it's somehow corrupt. PF has no way to figure out where these packets came from, or if they're maliciously damaged or just corrupted in transit. Since the kernel can't do anything with them, PF automatically drops them.

Packet Reassembly

Before the packet filter can decide how to handle a packet, the packet should be free of ambiguities and random weirdness. Reassembly cleans up these ambiguities, and the default reassembly settings are suitable for most environments. You get reassembly when you enable PF.

Packet Modification

Sometimes you need to modify packets. These days, PF handles everything for most environments. If you need to modify packets, such as clearing the "do not fragment" bit on fragmented UDP packets, see the `scrub` keyword in `pf.conf(5)`.

Blocking Spoofed Packets

Another classic IP attack is sending packets that appear to come from the private network to a firewall, in an attempt to evade the packet filter. Most firewalls today block this type of attack, so attackers rarely bother, but you should still protect against spoofed attacks. Just because everyone else has had their measles shot doesn't mean you should go without one.

For an antispoofing rule, use `antispoof for` and an interface name.

```
antispoof for fxp0
```

When fed into the packet filter, the rules would look something like this:

```
block drop in on ! fxp0 inet from 192.0.2.5/28
block drop in inet from 192.0.2.5 set ( prio 0 )
```

The first rule drops any traffic that arrives from an address local to interface `fxp0` on any interface other than `fxp0`. Packets from an address local to interface `fxp0` should always arrive on your system via `fxp0`.

The second rule drops any traffic that comes from the address of interface `fxp0`. Packets with that source address should never arrive from the outside world. If the system needs to communicate with itself, it uses interface `lo0`.

You could use interface groups instead of interface names, but I don't recommend doing so. If you have multiple `egress` interfaces, using antispoofing rules on the egress group won't block outside packets that arrive at the wrong egress interface. Take the time to enumerate your interfaces in your antispoofing rules.

Instead of listing a single interface, you can also use a list or a macro.

```
antispoof for {lo0, fxp0, em0}
```

Antispoofing rules can mess with packets passed over the loopback interface. I recommend skipping filtering on `lo0`, although PF includes special built-in protection for 127.0.0.0/8 addresses.

Now that you have basic packet filtering, let's consider some of PF's core settings.

PF Options

Options are basic settings that affect core PF functions. Options answer questions like these:

- Do we reassemble fragments into packets?
- How many entries should the state table support?
- Is logging on?

All options start with the `set` keyword. Because options affect how all other parts of PF operate, I recommend placing them at the very top of *pf.conf*.

Here, we'll look at some of the more commonly used options.

The set block-policy Option

Will your firewall silently drop forbidden packets, or respond to the client with "sorry, not allowed?" The block policy determines which approach it takes. By default, PF drops blocked packets, but you can override the global block policy on individual filter rules.

Strictly speaking, when PF drops packets, it should return an error to the client, so that legitimate clients can immediately recognize that they cannot connect. Using `set block-policy return` tells PF to return these polite errors: an RST for TCP connections and an ICMP unreachable message for other types of connections.

Unfortunately, politeness has largely been overwhelmed by the modern Internet. PF's default, `set block-policy drop`, tells PF to not return any kind of error on blocked packets. Client applications such as web browsers, vulnerability scanners, worms, and other malware must wait for the network protocol to time out before realizing that they cannot connect.

I recommend dropping blocked packets silently.[3]

The set limit Option

PF includes limits on the size of various internal tables used to track fragments, states, address tables, and other memory-consuming items. I have needed to adjust these limits on very rare occasions. The existing limits are chosen because they are sufficient for most users in most environments.

View the existing limits with `pfctl`.

```
# pfctl -s memory
states        hard limit    10000
src-nodes     hard limit    10000
frags         hard limit     1536
tables        hard limit     1000
table-entries hard limit   200000
```

Let's take a look at what each limit represents.

3. Mind you, if PF included an option to insult the client when a packet is dropped, somewhat like `sudo`, I would need to change my recommendation. But that's a fault in the underlying network protocol, not PF.

frags Limit

When PF receives a fragmented packet, it holds onto that fragment and waits for other fragments of that packet to arrive. Once it has all the pieces, it reassembles the fragment and processes it. The `frags` limit controls the number of packet fragments awaiting reassembly at one time. (You shouldn't need to change this.)

To see the total number of fragments PF has processed, and how many arrive per second, use `pfctl -s info` and look at the Counters section.

```
# pfctl -s info
...
  fragment                               368            0.0/s
...
```

This host has been sitting on the naked Internet for three months in an Internet colocation site, and has received only 368 fragments. I do not need to increase PF's memory for fragments, and I certainly don't want to reduce the limit in case I receive a sudden barrage of fragments.

If you suspect that fragments are flowing in, run `systat pf` for constantly updating counters of PF statistics.

The src-nodes Limit

PF can track a number of states per source address. You might want to limit each client to, say, 10 connections to a specific server. This connection limit includes connections being set up and those still waiting to finish. Here's an example of this sort of rule:

```
pass in proto tcp to $webserver port 80 keep state(max-src-states 10)
```

PF's load balancer features use `src-nodes` to help track which clients are attached to which servers, through the `sticky-address` and `source-track` options.

If you use these features, and think you might be out of source nodes, check usage with `pfctl -s Sources`.

The states Limit

The `states` limit controls how many entries can be in the stateful inspection list. The default of 10,000 is adequate for most environments.

You can view the current usage with `pfctl -s info`.

```
# pfctl -s info
Status: Enabled for 1 days 18:01:06             Debug: err

State Table                          Total             Rate
  current entries                       30
  searches                         54510751            6.3/s
  inserts                           2459724            0.3/s
  removals                          2459694            0.3/s
...
```

NOTE *I have needed to change the state table more than once. Each time, it was because of a strangely written application that required clients to make dozens of connections to a single TCP/IP port. I'm certain that the application developers had their reasons for doing so (possible reasons do include ignorance and malice). Multiplied by thousands of simultaneous users, that became a lot of states. As I wasn't in a position to tell the developers to write their application like normal people, I had to adjust the state table.*

If you suspect that the state table is having trouble, use `systat pf` and/or `systat states` to view state activity in real time.

The tables and table-entries Limits

The `tables` and `table-entries` limits control how many tables PF can create, and how many entries can go into a single table. I have never had to adjust these, and I would suggest that if your filter rules need more than 1000 tables, you should probably reconsider how you've designed it. A table might need to hold more than 100,000 addresses, but that's very much the exception these days.

Setting Limits

To change a limit, use `set limit`, the name of the limit, and the new value. Here's how to double the size of the default state table:

```
set limit states 20000
```

Again, don't change these defaults lightly. Increase them only if existing limits cause a specific problem. And don't decrease them, or you won't be prepared for problems and spikes.

The set optimization Option

PF includes a variety of timeouts, which default to values reasonable for the modern Internet. Some environments, such as satellite uplinks, do require slightly different timeouts.

You can adjust PF's timeouts with `set optimization`. (The name is a leftover from the early days of PF, but has stuck around.) This has four values:

`normal`
: The `normal` optimization is the default. If you don't specify an optimization, the standard timeouts are used.

`conservative`
: The `conservative` optimization is for environments where you want to be absolutely sure you don't time out connections. (State table entries will stick around longer.) This setting uses more memory and processor time—possibly much more on a busy network. I use it to ease the minds

of managers of industrial networks who are less concerned about buying more hardware and more concerned about the possibility of a meeting caused by some executive's idle connection timing out.

high-latency
: If you connect over a satellite uplink or carrier pigeon, use the `high-latency` optimization.

aggressive
: If you have a busy firewall, with many connections coming and going, you might try the `aggressive` optimization. This times out idle connections more quickly, reducing memory and processor use. Many people report that `aggressive` timeouts work perfectly well in their environments, but if low timeouts cause trouble for you, turn them off.

Configure any of these by using `set optimization` and the optimization name.

```
set optimization conservative
```

The set skip Option

You can tell PF to not manage an interface. By default, it watches all interfaces, but some interfaces don't really require filtering. Your loopback interface, `lo0`, passes traffic only from the local machine to itself. Packet filtering on `lo0` is an interesting educational exercise, but not terribly useful in production.

```
set skip on lo0
```

You can also specify multiple interfaces to skip.

```
set skip on {lo0 fxp0 fxp1}
```

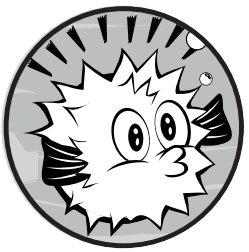
It's fairly common to skip filtering on the physical interfaces beneath a trunk in favor of filtering on the trunk itself.

This will get you started with packet filtering. If you have a single server with simple functions, you can protect it quite nicely using the techniques covered in this chapter. But PF can do a lot more than what we've talked about here, such as control bandwidth and have applications dynamically change rules. In the next chapter, we'll touch on a few of PF's more advanced functions.

22

ADVANCED PF

Office net seems slow
thanks to bootleg film swapping.
Let's stop that right quick!



The previous chapter covered the basics of the OpenBSD packet filter `pf(4)`. But, as I mentioned, PF can manipulate packets in all kinds of ways beyond just permitting or denying them, including the following:

- You can dynamically change the list of addresses to pass or block through outside software, such as `dhcpd(8)` or `spamd(8)`.
- You can dynamically create sub-rulesets that let you set up very specific rules for troublesome protocols without allowing more access than necessary.
- PF can provide NAT, letting you offer an entire network Internet access without public IP addresses.
- You can redirect incoming traffic arbitrarily, and control how much bandwidth you will let a service use.
- You can use PF logging.

This chapter covers each of these topics.

Packet Filtering with Tables

A *table* is a list of IPv4 and/or IPv6 addresses, much like a list. A table is faster than a list, however, and uses less memory. If you have only a few addresses, using a list is fine, but once you have more than a few, use a table.

Interestingly, you can edit tables without reloading the filter rules, and several programs use this feature to dynamically change how a server behaves. Some people load lists of malware-laden computers into a table to block those hosts, or use external programs to generate such lists. ("You've tried to send us four invalid emails in a row? Good-bye!") Tables can be kept permanently in external files, or you can treat them as ephemera. It's your choice.

Defining Tables

You can create and manipulate tables entirely with `pfctl`, but that's not as common as defining the table within *pf.conf*. Give the table name in angle brackets, and provide the initial members delimited by commas inside braces.

```
table <management> {192.0.2.5, 192.0.2.8, 192.0.2.81}
```

In this case, the `management` table contains three IP addresses.

If you want to define a table that `pfctl` cannot change, use the `const` keyword. The following example defines a table for private (RFC 1918) address space. This address space has been well defined for many years, so no one should alter it.

```
table <private> const {10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16}
```

If no rules reference a table, PF drops it. This makes sense for static rules, but if you're using anchors (discussed later this chapter), you might want to retain the table for when rules reappear. Use the `persist` keyword to make a table stick around even if it's not used in a rule.

```
table <scumbags> persist
```

Some tables contain enough addresses that you wouldn't want to list them in your configuration. For convenience, you can populate a table from a file, like this:

```
table <fullbogons> persist file "/etc/fullbogons.txt"
```

I have a script that updates the *fullbogons.txt* file every day. (*Bogons* are addresses that should never appear in the global Internet routing table.)

The bogons list includes private address space, addresses reserved for experimentation or documentation, addresses not assigned to any network, and addresses assigned to other exotic purposes. Several organizations

produce and update full bogon lists. I use the bogons list at my border to weed out obvious garbage. The file looks like this:

```
# last updated 1352220481 (Tue Nov  6 16:48:01 2012 GMT)
0.0.0.0/8
10.0.0.0/8
14.1.96.0/19
...
```

You can include individual addresses, but not dotted-quad netmasks. You can use hostnames, but before `pfctl` feeds the rules to the kernel, it checks the IP address or addresses of the host. This means that if a host changes its IP address after you load the rules, PF will not know about the new IP address.

Using Tables

Use the table in your firewall rules exactly as you would use an address or list.

```
block in on egress from <fullbogons> to any
```

You can put multiple tables in a list.

```
block in on egress from {<fullbogons>, <scumbags>} to any
```

Yes, a list is slower than a table. But if you maintain two different tables in different ways, you probably want those tables separated. And if a list of two items triggers firewall exhaustion, you really need more hardware.

Viewing Tables

Tables have their own subset of `pfctl` commands. To see which tables are in the kernel, use `pfctl -s Tables`. (Note that `Tables` begins with a capital `T`.)

```
# pfctl -s Tables
fullbogons
scumbags
```

Why would you need to ask the kernel what tables it has? Because dynamic rules can add and remove tables, as discussed in “Anchors” on page 434.

If you already know the table name, and you want to view the addresses within the table, use the `-t` argument to specify a table name. The `-T` argument has several subcommands, much like `-s`, but is for table operations. Here’s how to examine the contents of the `scumbags` table:

```
# pfctl -t scumbags -T show
   157.166.248.10
   157.166.248.11
```

```
157.166.249.10
157.166.249.11
```

For many table operations (`add`, `delete`, `replace`, and `test` as of right now), you can add one or two `-v` options before the `-T` to increase verbosity. If you work on multiple addresses simultaneously, adding verbosity shows details of what the command did.

Searching Tables

You can eyeball a table with four entries pretty easily, but if a table has thousands of entries, you won't want to page through it searching for an address. You could use `grep(1)`, but that can fail because an address might be part of a network that looks completely different. (I'm sure I *could* write a `grep` expression that matches 10.0.0.0/8 if I enter 10.99.61.4, but I don't want to try it.) You can test an address to see if it's in a table.

```
# pfctl -t fullbogons -T test 192.0.2.88
1/1 addresses match.
```

This address appears in the `fullbogons` table.

If you test multiple addresses in one command, use `-v` or `-vv` before `-T` to see which addresses match and which don't.

```
# pfctl -t scumbags -vvT test 192.0.2.88 198.51.100.90
1/2 addresses match.
M  192.0.2.88    192.0.0.0/22
198.51.100.90    nomatch
```

Using a single `-v` shows only matching addresses.

Changing Tables

One important feature of tables is that you can dynamically alter them without reloading the firewall rules. If you must add an address to a table, use `-T`'s `add` command.

```
# pfctl -t scumbags -T add 192.0.2.88
1/1 addresses added.
```

Add networks by specifying a netmask and multiple addresses in a single command.

```
# pfctl -t scumbags -T add 198.51.100.0/24 2001:db8::/32
2/2 addresses added.
```

If you add addresses to a nonexistent table, PF automatically creates the table (so now you know where that `scumbags` table came from).

Add all the addresses in a file to a table with the `-f` argument.

```
# pfctl -t scumbags -T add -f scumbags.txt
1/1 addresses added.
```

To remove addresses, use the `delete` command.

```
# pfctl -t scumbags -T delete  198.51.100.0/24
1/1 addresses deleted.
```

To completely remove all entries from a table, use `flush`.

```
# pfctl -t scumbags -T flush
6 addresses deleted.
```

If emptying the table is not enough, and you want to completely remove it from the rules, use `kill`.

```
# pfctl -t scumbags -T kill
1 table deleted.
```

Tables and Automation

OpenBSD includes software that can adjust tables algorithmically. In Chapter 16, I mentioned the DHCP server's ability to assign leased, abandoned, and changed addresses to tables. You can use PF to assign different rules to each group of addresses.

Assume you have `dhcpd(8)` add all leased IP addresses to the `leased` table, abandoned addresses to the `abandoned` table, and changed addresses to the `changed` table. Hosts with properly leased addresses can access the network, but hosts with abandoned and changed addresses cannot. Here, interfaces in the office group face the local network:

```
table <leased> persist
table <abandoned> persist
table <changed> persist
pass in on lan from <leased> to any
block in on lan from {<abandoned>, <changed>} to any
```

If someone decides to configure an address from the DHCP server as a static address for their computer, they automatically lose access to the rest of the network—problem solved. Other OpenBSD software, such as `spamd(8)`, has similar features.

At first glance, it might seem like this feature is ready for integration with other programs. It's fairly simple to write a script that parses a log, grabs the IP addresses, and feeds those addresses to a table. Several years ago, I wrote a script to take alerts from the Snort intrusion detection system and automatically block attackers from the network. Without careful and skilled

attention though, Snort generates many false positives. My autoblocking script very effectively created a denial-of-service attack against my own development team.

Be careful with automatically feeding PF tables to block traffic. It's very easy to harm desirable connectivity.

Using NAT

One of the critical functions of a firewall is NAT. Use NAT to provide IPv4 network access to multiple machines but show only one public IPv4 address. Some companies provide Internet access to thousands and thousands of machines via NAT.

NAT is like making soup out of a bone—it stretches what you have so that it covers more. Some protocols won't work well with NAT. It really confuses anyone who is trying to restrict access by IP address. And it can cause nightmares for network forensics and troubleshooters. But NAT is the chosen solution for the IPv4 address shortage.

NAT is *not* intended as a security mechanism. There are minor security benefits, but they are inadequate against today's network threats. Relying on NAT for security is chasing 10 boilermakers with a cup of black coffee before staggering out of the pub to drive home. You might get away with it, but only by luck.

IPv6 was designed without NAT, but it was shoehorned in several years later by popular demand. (IPv4 was originally designed without NAT as well, so IPv6 is just following tradition.) Note that an IPv6 address—even a globally unique IPv6 address—does not mean or even imply "reachable from the world." You can have solid network separation without NAT. Avoiding NAT means using your packet filter to protect your machines, with additional application proxies as needed.

Private NAT Addresses

In theory, you can use any addresses behind your NAT device. If you use some random IP addresses, though, you cannot exchange packets with whoever uses those IP addresses out in the real world. It's highly advisable to use some of the IP addresses reserved for private use, generally referred to as "RFC 1918 addresses." These include the following IP addresses:

- 10.0.0.0/8 (10.0.0.0-10.255.255.255)
- 172.16.0.0/12 (172.16.0.0-172.31.255.255)
- 192.168.0.0/16 (192.168.0.0-192.168.255.255)

You can subnet and rearrange those addresses any way you like, as long as you don't try to route them on the public Internet.

You can use other IP addresses behind your NAT if you have a really good reason for doing so. For example, RFC 5737 defines IPv4 addresses for use in documentation. Like RFC 1918 addresses, RFC 5737 addresses should never appear on the public Internet. I write documentation, so

I use those addresses on my home and test networks. It saves me from doing search and replace as I write books.[1] There's still no chance of those addresses appearing on other networks.

Configuring NAT

Perhaps the most common form of NAT is for use in hiding a small network behind a single IP address. You'll find this in many homes and small businesses. Very few home offices have internal routing and multiple subnets. For this example, I have two interface groups: the Internet-facing `egress` group and the `lan` group attached to my office.

```
pass out on egress from ❶lan:network to any ❷nat-to egress
```

The first part of this rule looks just like any other firewall rule permitting the addresses on the `lan` interface access to everywhere, but the last two words additionally configure NAT. The `nat-to` keyword tells PF to translate addresses ❷. The `egress` that follows tells PF to hide the internal addresses behind the addresses of the `egress` interfaces ❶. You could use an interface name or a specific IP address here, but if you do, you must change your filter rules when you change your server.

In order to have PF recognize IP address changes from DHCP, put the interface group name in parentheses.

```
pass out on egress from lan:network to any nat-to (egress)
```

Now load your firewall rules, enable IP forwarding, and suddenly, hosts on your LAN will have access to the Internet through the firewall's public address.

How NAT Works

The easiest way to understand how address translation works is to look at the state table (discussed in the previous chapter) after PF passes translated packets back and forth. On the office network from machine 192.0.2.2, I ran this command:

```
$ ping www.michaelwlucas.com
```

Several pings later, I checked the state table and found entries like this:

```
# pfctl -ss | grep 192.0.2.2
all udp ❶203.0.113.5:55797 ❷(192.0.2.2:10853) -> ❸203.0.113.15:53    MULTIPLE:SINGLE
all icmp 203.0.113.5:8813 (192.0.2.2:41584) -> 198.22.63.8:8       0:0
```

1. Can Lucas configure a highly available firewall cluster in a day? Yep. Can he search and replace IP addresses in a text file without screwing everything up? Nope.

The first state represents a UDP connection from the firewall's public address ❶ to the local DNS server ❸. This state entry includes the client's private IP address ❷, as well as the actual ports used by the client, the firewall, and the DNS server.

The client initiated this state by sending a request from port 10853 on its IP address to port 53 on the DNS server. When the packet passed through PF, OpenBSD rewrote the packet so that it appeared to come from the address 203.0.113.5 on port 55797 and sent it on to the DNS server. The DNS server sent its response to the firewall's public IP on port 55797. When the reply arrived, the firewall checked the state table, and found that UDP packets on port 55797 were part of the state for the client. PF rewrote the packet's destination address and forwarded it to the client.

The second state represents an ICMP connection. The state table encodes the various ICMP codes used for a ping request as port numbers, and forwards responses back to the client based on that information. Otherwise, it's very similar to the DNS example above it.

In other words, NAT works by lying. PF lies to the client, telling it that it has direct access to the public Internet. It lies to the external servers, giving a false source address and port for client connections. PF uses the state table to track its lies and keep everything consistent. These lies are convenient for IPv4 address conservation, but they're exactly why address translation complicates troubleshooting and intrusion forensics.

Now that you understand the basics of NAT, let's tell the network even more complicated and interesting lies.

Multiple or Specific Public Addresses

You can use several public IP addresses for address translation. If you use an interface group for the external address in your NAT rule, any addresses in that interface group can become the public address of any connection. If you want to be specific, list particular addresses.

```
pass out on egress from lan:network to any ❶nat-to 203.0.113.5
```

I use this configuration when my firewall's external interface has multiple IP addresses and I want to conceal my desktop clients behind a single address (although I probably would define and use a macro for the external address ❶).

But how many public addresses do you need? The answer depends on your clients.

Port numbers range from 0 to 65535. The bottom 1024 ports are generally used for services on the localhost. Not all of those ports will be used on the localhost, but a packet filter generally won't use those ports for translated connections. I'm lazy, so I'll round off to 64,000 free ports.

Even the most heavily loaded desktop client rarely can use as many as 100 outbound connections simultaneously. Most will use far fewer, but again, I'm lazy, and I want a worst-case scenario, so I'll call it 100.

One IP address can support 64,000 / 100 = 640 machines being pathological simultaneously. Realistically, each client might have 10 simultaneous outbound connections, so a public address could support 6,400 simultaneous clients. How many of your users browse the Internet at the same time? The answer probably is not many. And if you have thousands of users, you would probably benefit from implementing a caching proxy, which would greatly reduce the number of connections.

If you're concerned about overflowing the number of client machines for one address, watch your state table. Until you have multiple tens of thousands of states for one public IP address, don't worry.

Specifying individual addresses in a NAT rule is most useful for bi directional NAT.

Bidirectional NAT

Some applications work better if you dedicate a public IP address as the NAT address for a specific private IP address. For example, if you have a server that offers several different services on different ports, and you want to put it behind your firewall, you might want to dedicate a single address to it. This is called *bidirectional*, *one-to-one*, or *static* NAT. OpenBSD docs use "bidirectional," but the terms all mean the same thing.

Configure bidirectional NAT with the `binat-to` keyword.

```
pass on lan from 192.0.2.65 to any binat-to 203.0.113.6
```

PF dedicates the public IP address 203.0.113.6 for NAT services for the private IP address 192.0.2.65.

If you use bidirectional NAT, be sure to specify a specific IP address for your general NAT and consider using the following NAT rules:

```
pass out log on egress from lan:network to any nat-to egress
pass on lan from 192.0.2.2 to any binat-to 203.0.113.6
```

The IP addresses on this LAN are hidden behind the IP addresses on the `egress` interface. If 203.0.113.6 is an address on an `egress` interface, outbound packets from the LAN might use it as a source address.

When I need bidirectional NAT, I usually write my NAT rules like this:

```
mainnat="203.0.113.5"
servernat="203.0.113.6"
pass out log on egress from lan:network to any nat-to $mainnat
pass on lan from 192.0.2.2 to any binat-to $servernat
```

In this way, packets leaving my network are unambiguously translated. Only the one specific server uses the IP address 203.0.113.6; all other hosts on my local network use 203.0.113.5. If I change IP addresses, I must reconfigure *pf.conf*, but that's a minor annoyance compared to troubleshooting network ambiguity.

Bidirectional NAT and Security

The use of bidirectional NAT, and allowing the redirection of connections, lets you give people outside your network access to servers behind your firewall, and every one of these gaps is a potential security hole. If you allow the world access to your web servers, and an intruder compromises one of your servers, you have a compromised machine inside your firewall. The firewall doesn't really secure the web servers; it just controls who can try to break into them and limits the available attack vectors.

Packet Filtering, Bidirectional NAT, and Rule Order

When writing packet-filtering rules for bidirectional NAT, the order in which you list rules is important. Consider the following rules:

```
pass on lan from 192.0.2.2 to any binat-to 203.0.113.6
pass in on egress proto tcp from any to 192.0.2.2 port 80
```

The first rule establishes static NAT for the host 192.0.2.2 on the LAN, hiding it behind the public IP address 203.0.113.6. All is well and good. The second line permits connections to port 80 on the same host, or does it? Packets meant for this server that arrive on the firewall's egress interface won't be addressed to 192.0.2.2; they'll be addressed to the public NAT address, or 203.0.113.6. They won't match this rule, so they are discarded.

In order to permit connections from the world to the web server behind this firewall, permit packets sent to the proper port on the public address.

```
pass on lan from 192.0.2.2 to any binat-to 203.0.113.6
pass in on egress proto tcp from any to 203.0.113.6 port 80
```

This translates 192.0.2.2 to the public address 203.0.113.6, and then allows packets with a destination of port 80 on 203.0.113.6 to pass. You'll see this in the state table, like this:

```
all tcp 203.0.113.6:80 <- 198.22.63.8:64791       ESTABLISHED:ESTABLISHED
```

The host 198.22.63.8 has connected to the server's public IP address on port 80.

Why doesn't this state entry have the hidden IP address in it? Because this is a bidirectional NAT. PF can send port numbers through unaltered, so it can track a little less information in the state table.

The tricky thing here is that the rule order impacts how you filter, and you must read your filtering rules carefully to see how address translation interacts with packet filtering. I *always* write my rules so that I do address translation before I filter. I consistently use the public IP address in the filter rules, but sometimes that's not practical. PF lets you write arbitrarily complex rules mainly because the real world is arbitrarily complex. If you have trouble passing traffic through NAT, read your rules very carefully.

To see a bidirectional NAT, look at the loaded rules.

```
# pfctl -sr
...
pass out on lan inet from 192.0.2.2 to any flags S/SA nat-to 203.0.113.6 static-port
pass in on lan inet from any to 203.0.113.6 flags S/SA rdr-to 192.0.2.2
pass on egress inet proto tcp from any to 203.0.113.6 port = 80 flags S/SA
```

The first rule gives the private IP address access to the public Internet, translated to the specific IP address. The third rule passes traffic to the translated address.

But what about the second rule, with that `rdr-to` stuff? That's a redirection, which is how PF implements static NAT.

Redirection

Bidirectional NAT is actually a combination of address translation and *redirection*; in other words, it twists a connection intended for one IP or port to another. In bidirectional NAT, all connections to the designated public IP address are redirected to a different IP address. Sometimes you don't want to twist all traffic for an IP address—only a few ports. Sometimes you want to redirect one port one way, but a different port elsewhere. Do this with redirection rules.

Suppose you have one public IP address: 203.0.113.5. You want port 80 on that IP address routed to your web server at 192.0.2.2, ports 25 and 110 to your mail server at 192.0.2.3, and port 443 to your e-commerce server at 192.0.2.4. PF lets you choose where to send each port via redirection by using a standard packet-filtering rule and adding the `rdr-to` redirection keyword.

```
pass in on egress proto tcp from any to egress port 80 rdr-to 192.0.2.2
pass in on egress proto tcp from any to egress port {25, 110} rdr-to 192.0.2.3
pass in on egress proto tcp from any to egress port 443 rdr-to 192.0.2.4
```

These rules declare that any connection coming to the `egress` interface group (the interface facing the public Internet, with a default route going over it) can be redirected in three different ways. The first rule directs port 80 requests to one internal server. The second rule directs requests for ports 25 and 110 to the second server. The last rule redirects requests for port 443 to the third server. One public IP address is now providing services to the world from three different servers.

All port redirection rules must include a protocol, because specifying a TCP/IP port works only if you're forwarding a protocol that includes port numbers, such as TCP or UDP. If you want to forward both TCP and UDP ports, you must specify both protocols. For example, DNS uses port 53 on both TCP and UDP. Here's a rule that forwards both of these protocols' port 53 to the internal server 192.0.2.5:

```
pass in on egress proto {tcp, udp} from any to egress port 53 rdr-to 192.0.2.5
```

Pick a port, say where you want it to go, and PF will redirect it as you please.

NOTE *You've learned how bidirectional NAT combines redirection and address translation. The in-kernel PF engine doesn't actually know anything about this beastie called "bidirectional NAT." `pfctl(8)` translates the `binat` rule into two separate rules: one for translation and one for redirection.*

Multiple Addresses and Interface Groups

All of the preceding discussion makes sense when you have only one public IP address. But what happens when you have multiple addresses?

Remember that using an interface group in *pf.conf* tells `pfctl` to create a matching rule for every IP address in the interface group. Suppose you have three IP addresses on your `egress` interface: 203.0.113.5, 203.0.113.6, and 203.0.113.7. You write this *pf.conf* rule:

```
pass in on egress proto tcp from any to egress port 80 rdr-to 192.0.2.2
```

Load this rule into the kernel with `pfctl`, and what do you get?

```
# pfctl -sr
...
pass in on egress inet proto tcp from any to 203.0.113.5 port = 80 flags S/SA rdr-to 192.0.2.2
pass in on egress inet proto tcp from any to 203.0.113.6 port = 80 flags S/SA rdr-to 192.0.2.2
pass in on egress inet proto tcp from any to 203.0.113.7 port = 80 flags S/SA rdr-to 192.0.2.2
```

Any connection to port 80 on any of these IP addresses is directed to port 80 on the same server. This might be useful in some environments, but that's not what most of us want. If you have multiple IP addresses, and you want to redirect a port on only one IP address, you must specify the interface name and the public IP address.

```
pass in on em0 proto tcp from any to 203.0.113.5 port 80 rdr-to 192.0.2.2
```

This doesn't expand; it doesn't have any interface groups, lists of addresses, variables, or macros. When `pfctl` parses this, it loads only one PF rule into the kernel.

Port Manipulation and Ranges

As you redirect ports from one machine to another, you can change the port. The following example takes requests to TCP port 2222 on the firewall and redirects them to port 22 on a machine inside the firewall.

```
pass in on egress proto tcp from any to egress port 2222 rdr-to 192.0.2.2 port 22
```

This is a reasonable way to offer SSH services to several machines inside the firewall on only one IP address, and to give each machine its own port.

If you have specific source addresses that you want to abuse, you can give them special port redirections by source IP address.

```
pass in on egress proto tcp from 198.51.100.0/24 to egress port 80 rdr-to 192.0.2.2
pass in on egress proto tcp from ! 198.51.100.0/24 to egress port 80 rdr-to 192.0.2.3
```

Every HTTP connection from the IP addresses in 198.51.100.0/24 will be redirected to one server, while every other connection will be directed elsewhere. (To redirect connections for many source addresses, use a table for the source address.)

PF can also redirect entire ranges of ports using the same logical operators used for filtering ports. One obvious thing to do is to redirect a range of ports to a single machine. NFS is a prime example, as it requires TCP port 111, as well as all TCP and UDP ports from 1024 to 65535.

```
pass in on egress proto {tcp, udp} from any to egress port {111, 1024:65535} rdr-to 192.0.2.15
```

Recall from Chapter 21 that a colon between port numbers indicates a range of ports. This rule passes ports 1024 through 65535, inclusive. Admittedly, certain NFS implementations can be restricted to use either TCP or UDP, and that's a great big gaping hole in your packet filter. But NFS uses random high-numbered ports that come and go very quickly, and cannot be effectively filtered or restricted at the packet level.

You can also funnel an entire range of ports to one port on one machine.

```
pass in on egress proto tcp from any to egress port {1024:65535} rdr-to 192.0.2.15 port 80
```

I've used this to point random traffic at a web page that says "Go away. You cannot use this service."

Transparent Interception

Traffic interception is similar to redirection in that PF intercepts traffic bound for one port and steers it to a port on the local machine. Traffic interception is one way to implement a transparent proxy. Use the `divert-to` keyword to tell PF to steer any matching packets to a local server.

```
pass in inet proto tcp from lan:network to any port 80 divert-to 127.0.0.1 port 3129
```

Any traffic from the local LAN to port 80 will be diverted to port 3129 on the firewall. Port 3129 is usually used by the Squid caching proxy (*/usr/ports/www/squid*). If you choose to implement a caching proxy like Squid, you'll probably want to redirect several ports to the cache. (We'll take a closer look at diverting connections in "FTP and PF" on page 437.)

Anchors

In PF, an *anchor* is a sub-ruleset at a specific point in the filter rules that you can change without reloading the rules. It's a spot marked "insert rules here," letting you dynamically add and remove filter rules, tables, and other PF configurations.

The most common users of anchors are software programs. Human beings or sysadmins should probably just edit *pf.conf* and reload the rules.

OpenBSD includes several programs that take advantage of anchors, however, including the FTP proxy `ftp-proxy(8)`, the authenticated firewall access system `authpf(8)`, and the load balancer `relayd(8)`. You could also use anchors to trigger conditional evaluation of rules.

A ruleset with an anchor might look something like the following, where the interface group `egress` faces the Internet, and the interface group `lan` faces a small office with the addresses 192.0.2.0/24.

```
block
pass in on egress from any to 192.0.2.45 port {25, 80}
anchor "antivirus/*"
pass in on lan from 192.0.2.0/27 to any
```

These rules block all traffic by default. Incoming traffic is allowed to a specific address on ports 25 and 80 because those are the mail and web servers. There's an anchor in the middle of the rules. I don't yet know what's in the `antivirus` anchor, but any rules in it are processed next. Finally, a small subnet of the addresses is allowed out.

Now let's add some rules to the anchor.

Adding Rules to Anchors

You can insert rules into anchors from a file, within *pf.conf* itself, or via `pfctl`.

Anchor Rules from Files

Adding rules to an anchor from a file is a good way to initialize your anchor when first starting the packet filter. You can set base rules here that you can expand later. Give the filename in *pf.conf*.

```
anchor dhcp
load anchor dhcp from "/etc/pf/dhcp-anchor.conf"
```

I created an */etc/pf/* directory because I didn't want to have a whole bunch of PF configuration files scattered throughout */etc*. I'm easily confused, after all. This file contains PF rules like this:

```
block from 192.0.2.192/26 to any
```

This is one way to load basic rules into an anchor when you start PF.

If you were paying attention, you probably noticed that my first example anchor had a /* after its name. This example doesn't. I'll explain why in "Nested Anchors: /*" on page 436.

Anchor Rules in pf.conf

You can place anchor rules directly inside *pf.conf*. If you don't intend to dynamically alter the rules, you don't even need to name the anchor. Just use curly braces to define the beginning and end of the anchor.

```
anchor "smtp" on egress {
    pass proto tcp from 192.0.2.12 to any port 25
    }
```

This is just slightly more complicated than the anchors in the default *pf.conf*.

Why would you want to do this? Read "Conditional Filtering" on page 436.

Anchor Rules via pfctl

To dynamically alter anchor rules with `pfctl`, you need the name of the anchor and the rule you want to put in its place. For example, suppose I want to add a rule to the `antivirus` anchor in the first anchor example.

```
# ❶echo "block in from 203.0.113.8 to any" ❷| pfctl ❸-a antivirus ❹-f -
```

Let's look at this command slightly backwards. The `-a` argument to `pfctl` specifies an anchor name—in this case, the `antivirus` anchor ❸. The `-f` argument normally gives a filename that contains the new anchor rule, much like `-f` when loading a PF ruleset, but rather than a path to a file, I use a single dash that tells `pfctl` to read the new rule from standard input, or the command line ❹. I start everything by echoing the rule to be added ❶, and then piping that into `pfctl` ❷.

Taken as a whole, this adds the rule `block in from 203.0.113.8 to any` to the anchor `antivirus`.

You could also write the new rule to a file, and tell `pfctl` to load the rules from that file into the anchor.

```
# pfctl -a antivirus -f newrule.conf
```

If you're writing rules to a file to load them into an anchor, however, chances are you're better off editing *pf.conf*.

NOTE *Adding a rule to an anchor erases any rules already in the anchor. If you have a software package that updates anchor rules, your software needs to handle this behavior. If your desired behavior can be accomplished using a list of IP addresses, consider using a table instead of an anchor.*

Viewing and Flushing Anchors

Use the `pfctl` view (`-s`), flush (`-F`), and load (`-f`) commands on anchors by specifying the anchor name with `-a`.

```
# pfctl -a antivirus -s rules
block drop in inet from 203.0.113.8 to any
```

To erase the rules from an anchor, flush the rules in the anchor.

```
# pfctl -a antivirus -F rules
rules cleared
```

Your anchor is now empty.

Rulesets within anchors are completely separate from each other, and also from the main ruleset. Flushing all the rules in a specific anchor does not affect the rules in any other anchor, or the rules in the main ruleset. For that matter, flushing the rules in the main ruleset does not impact the rules in the anchor. To destroy an anchor, you must remove everything in the anchor, including any child anchors.

"Child anchors?" I hear you cry. "What are you babbling about *now*, dude?"

Conditional Filtering

Consider the following *pf.conf* snippet:

```
...
anchor "office/*" in from lan to any {
    pass out proto tcp from any to {80, 443}
    }
...
```

The `office/*` anchor has a filter condition after it, and only traffic that matches the filter condition will pass through the anchor. In this case, only packets that come from the `lan` interface group will pass through the rules within the anchor. Your rules within the anchor might be easier to write, simply because everything in the anchor is already known to be originating from the `lan` interfaces.

If your packet filter is very heavily loaded, you might be able to reduce the amount of time it spends processing packets by careful conditional filtering.

Nested Anchors: /*

Anchors can contain other anchors.

```
anchor "office" in from lan to any{
    ...
    anchor "ftp-proxy/*"
    pass in quick inet proto tcp to port ftp divert-to 127.0.0.1 port 8021
}
...
```

Only traffic that passes into the `office` anchor can pass through the `ftp-proxy` anchor. The FTP proxy can have its own sub-anchors as well. In fact, you might have several layers of anchors to support a complicated protocol, such as FTP.

This is where the `/*` after some anchor names comes in. An anchor name without this is executed all by itself. By adding the `/*`, you tell PF to evaluate all sub-anchors within this anchor, in alphabetical order.

Anchors and sub-anchors deliberately resemble a filesystem. You can have a file */office* or a directory */office/* containing more files. If you list the files in a directory, they appear in alphabetical order. Anchors work much the same way.

All of this anchor stuff is very theoretical. How about a practical example? Read on to see how PF uses anchors to handle that most annoying of network protocols: FTP.

FTP and PF

Most modern application protocols run over a single network connection. If you make a web request, your browser opens a connection to the server on port 80, requests information, and receives the answer, all on the same connection. SSH opens a single connection on port 22 and exchanges all information over that port, even if you tunnel a hundred other protocols inside it. Experience and experiments with older protocols taught the wisdom of this approach. FTP is an older protocol, and it provides a wealth of experience on how not to do things.

The original version of FTP (today called *active FTP*) required the client to connect to the server on port 21. The server would then open a connection back to the client, from port 20 to some random high-numbered port on the client for sending information. The connection from server to client is called the *data connection*, or the *back channel*. The FTP client and server agree on the ports to be used and how the second connection will be used. On a network protocol level, however, no connection exists between the client's connection to port 21 and the server's connection from port 20, so there's no way for a firewall to use stateful inspection to sort out if such a connection is allowed. Worse, if the client is behind a NAT device, there's no way to determine to which private IP address the firewall should route an incoming FTP data request.

Passive FTP is an updated version of the FTP protocol where the client initiates both TCP connections. All modern clients and servers support passive FTP. The differences between active and passive FTP spark endless rounds of user education and increased help-desk load, especially if you're trying to use FTP through a web browser. (And if anyone is going to break my help desk staff, it's going to be me!) Active FTP simplified firewall rules, because the firewall didn't need to allow the back channel. Unfortunately, the creators of passive FTP called the modified protocol FTP. Clients don't care about active or passive, they just want "this FTP thing" to work, regardless of the actual protocol underlying it.

To complicate things, some FTP servers and clients implement something between active and passive FTP. The FTP protocol has been around for decades (it predates TCP/IP), and people have tweaked and "improved" it for years. Getting a random combination of FTP server and client through a random NAT device and a packet filter can cause nightmares, or at least require opening a wide range of TCP ports.

OpenBSD and PF get around this problem by including an FTP application proxy, `ftp-proxy(8)`. When a client makes an FTP request, PF intercepts the request and reroutes it to the application proxy. The proxy tracks the FTP protocol transactions, uses anchors to insert the appropriate rules into the firewall, and removes the rules when the transfer finishes. Strictly speaking, `ftp-proxy` isn't a traditional proxy. Data doesn't actually go through `ftp-proxy`; the "proxy" adjusts the firewall rules so that traffic can pass. The proxy requires two parts: a running `ftp-proxy` instance and the redirect rules.

Configuring ftp-proxy(8)

Like any other OpenBSD daemon, `ftp-proxy` is enabled in */etc/rc.conf.local*. There's no configuration file—only command-line arguments. By default, `ftp-proxy` automatically listens on port 8021 on the loopback interface. It's very rare for me to add any command-line arguments for `ftp-proxy` for routine use.

```
ftpproxy_flags=""
```

If I'm debugging a problem, however, I might run `ftp-proxy` in the foreground, in debugging mode. Doing this shows me all FTP transactions as they occur.

```
# ftp-proxy -dD7
```

This displays everything that passes through the FTP proxy, including the ports used for the data channel back to the client. Press CTRL-C to stop `ftp-proxy`.

The most common problem I have with `ftp-proxy` is that nothing appears in the debugging terminal. That means that the firewall isn't diverting any traffic to the proxy. Check your *pf.conf* file to verify that you have the necessary rules to support the FTP proxy.

PF Configuration and the FTP Proxy

PF must know to send FTP requests to `ftp-proxy`. There's a good example configuration in the default *pf.conf* file:

```
anchor "ftp-proxy/*"
pass in quick inet proto tcp to port ftp divert-to 127.0.0.1 port 8021
pass out inet proto tcp from (self) to any port ftp
```

Here's where we use anchors. The `ftp-proxy/*` anchor can contain subrulesets. The `ftp-proxy` daemon modifies these anchors on the fly to configure the necessary traffic or data connections. The second rule declares that PF will divert any traffic addressed to the FTP port (21 as per */etc/services*) to port 8021 on the localhost. The third rule says that the firewall host can send TCP port 21 traffic to any other host. This rule contains a new term, (`self`), which is PF shorthand for "all IP addresses on the localhost."

How can you be sure this works? First, find an FTP server that supports active FTP. Open your FTP client and log in to the server, going through the firewall. Once you log in, use the `pasv` command at the FTP prompt. This command turns passive mode on and off. If the server doesn't recognize `pasv`, it supports only passive FTP. Find another FTP server for this test. Once the FTP server reports that "passive mode is off," list the contents of a directory. Directory listings, like data files, come over the data channel.

During the data transfer of an active FTP connection, you should see rules in the `ftp-proxy/*` anchor.

```
# pfctl -a "ftp-proxy/*" -sr
anchor "6837.2" all {
  pass in log (all) quick on rdomain 0 inet proto tcp from 129.128.5.191 to
139.171.202.34 port = 62323 flags S/SA keep state (max 1) rtable 0 rdr-to
192.0.2.2 port 64280
  pass out log (all) quick on rdomain 0 inet proto tcp from 129.128.5.191 to
192.0.2.2 port = 64280 flags S/SA keep state (max 1) nat-to 129.128.5.191
}
```

The rules created by `ftp-proxy` are very specific. They permit only one connection, from a particular server to a particular client, with address translation rules to make each side think it's actually talking to the proper client or server.

NOTE *To learn how to restrict your clients to using only anonymous FTP, or how to use `ftp-proxy` to permit inbound FTP access to a server inside your firewall, read the `ftp-proxy(8)` man page.*

Bandwidth Management

One common task for a network perimeter device is bandwidth management. Network managers must control how much bandwidth is used for certain tasks, and must also reserve bandwidth for vital functions. If one of your minions loads the latest blockbuster comic book movie on the web server, you must be able to make an SSH connection to the server, find out why your server is overloaded, and fix the problem. PF includes the ALTQ bandwidth management system.

The most important thing to remember about bandwidth management is that you cannot control how much traffic other people send you. You can stop traffic at the point it enters your network. You can send hints that the

bandwidth is saturated. You can arbitrarily restrict bandwidth *from* your servers. But nothing you do can stop 10,000 people a second from clicking a link to that server. You cannot prevent a distributed denial-of-service attack from saturating your inbound bandwidth. The best you can do is control how you respond to those requests.

When I run content farms, I usually put dedicated bandwidth control machines in front of my servers. This setup controls how much traffic actually reaches my server network, reduces load on the servers in case of a sudden spike, and prevents one overly busy customer from taking down other customers on the same server.

Queues for Bandwidth Management

ALTQ manages bandwidth by *queues*. A queue is a list of packets waiting to be processed.

By dividing your bandwidth into separate queues, and processing those queues as you configure, you can manage server bandwidth. Queues are somewhat like the checkout lines at the grocery store; some lines are for 10 packets or less and get you out quickly, and others are for people who shop once a month and fill up three carts. You can define just about any characteristics for queues, as if you could create a "meats only" or "white wine with fish" register.

Engineers have defined many different queuing algorithms, and the most proper queue method for a given situation is a topic that sparks heated discussions. TCP/IP quality-of-service queue handling is one of those topics that make angelic children cry. By default, all BSD-based systems use first-in, first-out (FIFO) queuing, where packets are processed in the order in which they are received. Newer packets wait in a queue until older packets move on.

OpenBSD also supports priority queuing (PRIQ or prio), where the kernel considers packets of certain types to have "priority" and processes them first. This means that if you assign web packets highest priority, all web packets jump to the head of the queue. Packets of lower priority might never be processed at all under this scheme. These days, just about everything supports priority queuing, especially switches. The goal of priority queuing is to reduce latency for specific traffic, such as voice or video, paying for that reduced latency by increasing the latency of less urgent traffic.

However, in most operational settings where you must regulate bandwidth, class-based queuing (CBQ) is appropriate. CBQ allows the network administrator to allocate a certain amount of bandwidth to different types of traffic through hierarchical classes. Each class has its own queue, with its own bandwidth characteristics. You can assign different sorts of traffic to different classes: SSH to one class, HTTP and HTTPS to another, and so on. One of the nice features of CBQ is that its hierarchical nature allows lower classes to borrow available bandwidth from classes above them.

As I find CBQ appropriate for most environments, I focus on it here. Once you master CBQ, if you need PRIQ, you'll find it easy to understand.

Parent Queue Definitions

Queuing starts with defining the parent queue. All other queues are children of the parent queue. The parent queue is attached to a network interface, most commonly the Internet-facing interface. Place your queue definitions in *pf.conf*. I put queues at the top of the file, before any packet-filtering rules.

Here's how you define a parent queue on an interface:

```
❶altq on ❷interface ❸cbq bandwidth ❹bw qlimit ❺qlim tbrsize ❻size ❼queue { ❽queue1, ❾queue2}
```

Start all ALTQ parent queue definitions with the `altq` keyword ❶, and then give the interface to which this queue is attached ❷. (Each interface can have no more than one parent queue.) Then give the queue type you're using ❸. For CBQ queuing, the queue type is always `cbq`.

Now define the total amount of bandwidth in the parent queue ❹. This is not the same as the amount of bandwidth the interface can pass, but the amount of bandwidth you reasonably expect to pass upstream. If your OpenBSD machine has a gigabit network card, but you have only 10 megabits of bandwidth to the Internet, use `10Mb` as your bandwidth (or fiddle with the bandwidth value until you hit your actually usable allocation). You can use the following case-sensitive abbreviations for bandwidth:

`b` bits per second

`Kb` kilobits per second

`Mb` megabits per second

`Gb` gigabits per second

The optional `qlimit` parameter gives the number of packets the queue can hold ❺. The default value is `50`, which suffices for almost all cases. I recommend not setting `qlimit` unless specific debugging shows that you need a larger queue size.

This example includes the token bucket regulator size configuration because `tbrsize` lets you dictate how quickly packets can be transmitted ❻. ALTQ defaults to transmitting packets as fast as the wire permits. As with `qlimit`, I recommend not setting `tbrsize` unless you encounter a problem.

Next, identify this as a parent queue ❼, and define child queues `queue1` ❽ and `queue2` ❾.

Here's how to configure a parent queue with a 50-megabit uplink, with the child queues `ssh`, `web`, and `mgmt`:

```
altq on em0 bandwidth 50Mb queue {ssh, web, mgmt}
```

The `tbrsize` and `qlim` keywords are not set, so they're at their defaults.

Child Queue Definitions

Once you have a parent queue, you can define child queues. Define CBQ queues with the following syntax:

```
queue ❶name on ❷interface bandwidth ❸bw [priority ❹pri] [qlimit ❺qlim] cbq
❻(options) ❼{child_queues}
```

Each queue needs a name ❶, defined in the parent queue definition, of 15 characters or less. The names don't need to be unique—you could use a queue of the same name on a different interface—but I recommend that you use unique names.

The interface is the specific interface to which this queue is applied ❷. If you don't define an interface, traffic that passes through any interface can be assigned to this queue.

The `bandwidth` term uses the same bandwidth labels that the parent queue uses, but the total bandwidth assigned to all child queues cannot exceed the total amount of bandwidth available on the parent queue ❸. You can also use a percentage value for bandwidth, indicating the percentage of the parent queue that this queue can consume. Bandwidth and queue are the only mandatory terms in a child queue description.

The following defines the `ssh` child queue and gives it a bandwidth of 2 megabits:

```
queue ssh bandwidth 2Mb
```

Here's a child queue called `web`, which is allowed to use three-quarters of the parent queue bandwidth:

```
queue web bandwidth 75%
```

You can assign a priority to a queue ❹. CBQ priorities run from `0` to `7`, with `7` being the highest. The default priority is `1`. A CBQ queue with a higher priority does not run to the exclusion of other queues, but PF processes it more quickly than other queues.

As with a parent queue, you can assign a `qlimit` to a child queue ❺, but don't do this unless you have a specific problem that can be solved with this value.

You can assign options to a CBQ child queue ❻. We'll look at these options in the next section.

Finally, child queues can have their own children. Define a queue's children in the queue ❼. You'll see an example of this in "A CBQ Ruleset" on page 443.

Queue Options

Modify how a child queue processes packets by assigning options to a queue. Options let you decide how the queue should respond to a variety of network conditions and bandwidth availability.

Default

Every parent queue must have one and only one default child. If a packet crossing a queued interface is assigned to no other queue, it is assigned to the default queue.

Random Early Detection

Random early detection (RED) is a method for handling packet loss when a queue starts to fill up. As the queue fills up, more and more packets are dropped. RED randomly chooses packets to drop. The net effect is that short transfers, such as HTTP requests and interactive SSH sessions, respond more quickly, while large data transfers become slower.

TCP clients and servers react to dropped packets by reducing their throughput. UDP, ICMP, and other protocols don't have any built-in reaction to packet loss. Using RED on queues expected to carry TCP is sensible, but not on queues for other protocols.

Explicit Congestion Notification

Explicit Congestion Notification (ECN) is a modification to RED that sets flags in the packet rather than dropping the packet. If a device recognizes the ECN flag, it will reduce transmission rates.

Not all platforms understand ECN, however, and many that can recognize ECN disable it by default. Microsoft's Windows Vista and newer, Apple OS X, FreeBSD, and OpenBSD can support ECN, but disable it by default. Newer Linux versions support ECN if the other host requests it. I have successfully used ECN, in corporate environments where I could make the support guys enable ECN on the desktops.

Unless you know the operating systems in use and can control their settings, stick with standard RED.

borrow

The `borrow` option is available only in CBQ. A queue with borrow set may borrow bandwidth from its parent queue, if the bandwidth is available. For example, you might have a queue that reserves 20 percent of your bandwidth for VoIP. If you don't have that much VoIP traffic at any particular moment, the parent will have excess bandwidth. Other queues could borrow bandwidth from that allocation. When your VoIP traffic spikes, however, PF revokes the bandwidth loan, and the VoIP traffic gets what's reserved for it.

Use the `borrow` option on the queues that you want to permit to borrow bandwidth, not on the queues whose bandwidth might be borrowed.

A CBQ Ruleset

Before configuring queues, figure out how you want to divide your bandwidth. While you could use bits per second to manage bandwidth, for most of us, percentages are easier to deal with. Here's how you might divide Internet bandwidth for a company with a 10-megabit link. Start by making

a list of your desired bandwidth reservations, and then assign a name to each category, like this:

- 5 percent for SSH (`ssh`)
- 50 percent for inbound traffic to our e-commerce server, with RED (`web`)
- 5 percent for inbound VoIP, high priority (`voip`)
- 40 percent for other traffic, including DNS, SMTP, and so on

All of these queues can borrow from the parent queue.
Start by defining the parent queue.

```
altq on em0 cbq bandwidth 10Mb queue {ssh, web, voip, other}
```

This parent queue is attached to interface `em0`, and has 10 megabits of bandwidth and four child queues. Leave all the other options alone.
Now define the first child queue.

```
queue ssh bandwidth 5% cbq (borrow)
```

Start with the queue name and the bandwidth percentage you've chosen. This percentage is calculated from the parent of this particular queue, so it's about 5 percent of 10 megabits, or 500 kilobits per second. That should be plenty to log in remotely and fix any problems. Adding the `borrow` option lets you use more bandwidth for SSH if it's available.
Building from this example, you can define the other child queues.

```
queue web bandwidth 50% cbq (borrow, red)
queue voip bandwidth 5% cbq (borrow)
queue other bandwidth 5% cbq (borrow, default)
```

The other queue is your default. Any traffic that isn't assigned its own queue is assigned to this queue.

Assigning Traffic to Queues

Assign traffic to a queue with the `queue` keyword at the end of a packet-filtering rule. To allow all SSH (port 22) traffic into the network and assign it to the queue named `ssh`, use a rule like this:

```
pass in on egress proto tcp from any to lan:network port 22 queue ssh
```

Using the match Keyword

Sometimes you must classify traffic without filtering it. The previous example let you assign inbound SSH traffic to the `ssh` queue, but what if you want to capture outbound SSH as well? Consider the following rule snippet:

```
pass in on egress proto tcp from <customers> to <sshservers> port 22
pass out on egress from lan:network to any
```

This allows hosts in the customers table to connect to hosts in the sshservers table on port 22. The second rule allows the local network to send any traffic, or any protocol. Some of that outbound traffic will be SSH traffic. Should you write a separate rule just for queuing traffic?

This is where the match keyword comes in. Using match, you can change how PF classifies traffic without changing how it filters traffic. Here's how to send all TCP port 22 traffic to the ssh queue, without changing any filtering characteristics:

```
match proto tcp from any to any port 22 queue ssh
pass in on egress proto tcp from <customers> to <sshservers> port 22
pass out on egress from lan:network to any
```

The first rule matches all traffic on TCP port 22 and assigns it to the ssh queue. The rules that follow control who can send and receive SSH connections.

Viewing Queues

To view the queues currently in the packet filter, run pfctl -s queues.

```
# pfctl -sq
queue root_em0 on em0 bandwidth 10Mb priority 0 cbq( wrr root ) {ssh, web, voip, other}
queue  ssh on em0 bandwidth 500Kb cbq( borrow )
queue  web on em0 bandwidth 5Mb cbq( red borrow )
queue  voip on em0 bandwidth 500Kb priority 7 cbq( borrow )
queue  other on em0 bandwidth 500Kb cbq( borrow default )
```

Adding -v gives you a brief snapshot of the state of each queue. For a constantly updating view of all queues, including how much traffic is borrowed from each, what gets dropped, and so on, use -vvsq or systat queues instead.

PF Edges

This section covers a couple tidbits of PF configuration that don't quite fit anywhere else: include files and the quick keyword.

Using Include Files

Sometimes splitting a configuration file into multiple pieces simplifies your work. Do this with an include statement in *pf.conf*.

```
include "/etc/pf/management-addresses"
```

I do this when I need to manage several PF machines with unique configurations, but certain pieces are identical. The *management-addresses* file defines a table listing all hosts and networks that can connect via SSH, make SNMP queries, as so on. When one of those addresses change, I copy this file to all of my PF hosts and reload the packet-filtering rules.

Skipping Matches with quick

PF processes packet-filtering rules in order, and the last matching rule wins, which can complicate designing a ruleset that supports exactly the access you desire. If you find yourself stuck, use the `quick` keyword to abort processing the rest of the rules for matching packets. Here's an example:

```
...
pass in quick proto tcp from any to $sshserver port 22
...
block in proto tcp from any to any port 22
...
```

The first rule permits traffic to the host(s) in the macro `$sshserver` on port 22. The second rule drops all TCP port 22 traffic. The `quick` keyword in the first rule says, "When a packet matches this rule, follow this rule and do not process any more rules." In this case, the SSH connection will be permitted.

The `quick` keyword is especially useful in anchors, where rules added for a special purpose by an automated process like `ftp-proxy(8)` might be overridden by later rules meant for unrelated purposes.

The purist in me wants to insist that all static rulesets be written without using `quick`. While strictly speaking that's true, sometimes avoiding `quick` creates rulesets that are difficult to interpret. A ruleset you can easily understand is more secure than something baroque but syntactically pure.

Logging PF

Tell PF to log packets with the `log` keyword in a rule.

```
pass out log on egress from lan:network to any
```

Without additional setup, however, those logs just go to the PF log device `pflog0`. To successfully log PF messages, you must run the packet filter logger `pflogd(8)`. If you start PF at boot, `pflogd` is automatically started with it. Otherwise, you must start it on the command line.

One thing to remember is that if you're using stateful inspection, only the first packet that triggers a rule is logged. Other packets that are part of the same state are not logged. To log all packets in a stateful connection, give the `all` modifier to the `log` keyword, but beware because this can generate very large logs.

```
pass out log (all) on egress from lan:network to any
```

Logging is especially useful when troubleshooting connection problems. If packets are being blocked when you think they should be passed, add logging to your `block` statements to see which rule is stopping the traffic.

I don't recommend logging everything, especially because logs can grow quite large. Log selectively. For example, perhaps you don't care which

websites your local users visit, but do want to know about incoming traffic. And be sure to exclude your firewall logging traffic from your packet filter logs, or you'll quickly find that PF is logging the transmission of the logs of the log transmissions, which are logs of transmitting the logs, from when you transmitted the logs . . . yadda yadda yadda.

Reading PF Logs

PF logs in the `tcpdump(8)` binary format. Use `tcpdump` to examine the data. To just dump everything in the log, tell `tcpdump` to read the log file.

```
# tcpdump -r /var/log/pflog
```

This can generate a huge amount of output. See "Filtering tcpdump" on page 447 for some hints.

Real-Time Log Access

The entries in */var/log/pflog* are not added in real time; `pflogd(8)` buffers its records until writing a log message is worthwhile. To see PF logs in real time, attach `tcpdump` to the `pflog0` interface with the `-i` flag.

```
# tcpdump -i pflog0
```

Depending on how much traffic you're logging, this might also produce an overwhelming amount of information. You must filter `tcpdump` to make it useful. Or if you pretend you missed my earlier warning about log sizes, you can devise a one-liner that uses `logger` to send your PF logs as text to `syslog`.

Filtering tcpdump

Every system administrator should know how to use `tcpdump`. Here's your motivation for doing so.

When troubleshooting a problem with a particular connection, you probably don't care about every packet passing through the filter. You care about traffic to or from a particular host. Specify an IP address with the `ip` or `ip6` expression.

```
# tcpdump -i pflog0 ip host 192.0.2.2
```

This will display only traffic to and from this particular host.

To narrow things further and see only the traffic between two hosts, combine the hosts with the `and` keyword.

```
# tcpdump -i pflog0 ip host 192.0.2.2 and ip host 203.0.113.88
```

Maybe you're interested in only a specific port, on a specific address. Use the `tcp` or `udp` keyword and the port number to filter on that.

```
# tcpdump -i pflog0 ip host 139.171.199.254 and tcp port 80
```

Read the tcpdump(8) man page for an exhaustive list of innumerable other filtering options.

If using tcpdump doesn't appeal to you, consider the pflow(4) NetFlow exporter. Network flow is a complicated topic, but the book *Network Flow Analysis* (No Starch Press, 2010) might help you.

Ruleset Tracing

Sometimes, knowing whether a packet passed or failed isn't enough. You know that a packet was blocked, but not why. You want to watch the packet pass through the rules and see which rules affect it.

Suppose an internal host 192.0.2.226 cannot connect to the external host 203.0.113.34. The log would show that the packet is blocked, but not why. You can specifically have PF log matching rules. Add a line like this to the top of your *pf.conf* file:

```
match log (matches) from 192.0.2.226 to 203.0.113.34
```

This is a standard packet-filtering rule. You could use an individual IP address, a port number, or any other legal packet filter terms. Reload your packet-filtering rules.

Turn on tcpdump, and filter based on one of the IP addresses in your match statement. If you're using NAT, filter on the IP address that doesn't change.

```
# tcpdump -n -e -ttt -i pflog0 ip host 203.0.113.34
Dec 17 18:05:07.773703 rule 0/(match) match out on fxp0: 192.0.2.226.24916
> 203.0.113.34.22: S 1730871963:1730871963(0) win 16384 <mss 1460,nop,nop,
sackOK,nop,wscale 3,nop,nop,timestamp 597858150[|tcp]> (DF)
Dec 17 18:05:07.773708 rule 2/(match) block out on fxp0: 192.0.2.226.24916
> 203.0.113.34.22: S 1730871963:1730871963(0) win 16384 <mss 1460,nop,nop,
sackOK,nop,wscale 3,nop,nop,timestamp 597858150[|tcp]> (DF)
Dec 17 18:05:07.773712 rule 5/(match) pass out on fxp0: 192.0.2.226.24916
> 203.0.113.34.22: S 1730871963:1730871963(0) win 16384 <mss 1460,nop,nop,
sackOK,nop,wscale 3,nop,nop,timestamp 597858150[|tcp]> (DF)
```

While I won't go through all the annoying details of reading tcpdump output, you can see that PF logs the rule numbers that this data connection matches, and whether the rule passes or blocks the connection. If the connection involves NAT, you'll see the actual and translated IP addresses.

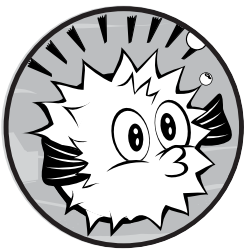
At this point, you know enough about PF to protect a small network. If you need more, definitely check out *The Book of PF, 2nd edition* (No Starch Press, 2010).

Now let's look at some of the more exotic edges of OpenBSD.

23

CUSTOMIZING OPENBSD

Customize installs
with files and DHCP,
then run without disk.



This chapter covers different ways to use OpenBSD to customize itself, as well as how to install OpenBSD in nonstandard situations and debug problems with your system. The first task we'll address is diskless installation. Diskless systems are usually used to install OpenBSD without attaching any installation media, but they can also be used to run a system without a hard drive. Next, we'll create a USB flash drive for use as OpenBSD installation media. Finally, we'll cover various ways to customize the OpenBSD installation and upgrade processes.

All of these tasks assume that you already have an OpenBSD machine running the version you want to customize. You can accomplish some of these tasks using a virtual machine, as long as the virtual machine software has the necessary support. Because virtualization is such a common choice, let's tackle it first.

Virtualizing OpenBSD

The OpenBSD developers are pretty clear on virtualization. OpenBSD is written for real hardware. Virtual hardware is not real hardware. While it can be very similar, it's not exactly the same.

This approach has a number of implications, the most problematic of which is that not all virtualization software can run OpenBSD. As I write this, Oracle's VirtualBox can't cleanly run either i386 or amd64 OpenBSD. (Some people report being able to boot some versions of VirtualBox and/or OpenBSD, but OpenBSD software crashes all over the place.) This is not an OpenBSD bug. VirtualBox doesn't sufficiently emulate real hardware.

That said, OpenBSD does run well on some virtual machines. VMware works well enough that OpenBSD includes specific drivers for VMware integration, including a VMware Tools driver in the kernel. KVM virtualization also works, although KVM requires some tweaks depending on the exact combination of KVM and OpenBSD you're using. Microsoft's virtualization mostly works, although Virtual PC has some commercially motivated limitations.

The main problem with virtualization is that a compromise of the virtualization platform automatically gives an intruder hardware-level access to all virtual machines, and OpenBSD cannot possibly secure you against that kind of attack. In fact, no operating system can. And it does you no good to run your database on OpenBSD when any script kiddie can compromise the underlying virtualization server.

In my experience, OpenBSD virtual machines are excellent for experimentation and reference. I used them to document the installation process for this book, and I always test software configurations on virtual machines before rolling them out to production. (The real benefit of virtualization might be that there's no longer any excuse for not testing changes.) But when I want a server that's actually secure, I put OpenBSD on real hardware.

NOTE *If you want to run virtual machines on OpenBSD, you can find* `qemu`, `bochs`, `dosbox`, *and other packages in the packages collection. Check* /usr/ports/emulators *for other options.*

Diskless Installation

Booting a blank system into the OpenBSD installer without using local media can save you time and energy. A lot of modern hardware doesn't come with CD or floppy drives. Of course, you could temporarily add a CD drive, but if you have a whole bunch of OpenBSD machines to install, that's just an annoyance.

You can also use network booting to boot OpenBSD on hardware that lacks an installed operating system, or with a different operating system that you plan to overwrite. This process is called *pxebooting*, or *diskless*, operation. Diskless systems can have disks—they just don't use them to boot the operating system.

If you've never worked with diskless systems before, your first attempts will probably give you a headache. Setting up your first diskless environment can be tricky, and will teach you all sorts of things you didn't know about your operating system and hardware. But test everything along the way, read the error messages carefully, and soon you'll wonder why you thought this was hard.

NOTE *I'll cover diskless installations on amd64 and i386 hardware. Other platforms have different requirements that may be very different. Read the* `diskless(8)` *man page for your particular architecture to get an overview of your platform.*

Diskless systems work because a computer doesn't need a hard disk to run. It needs an operating system. The easiest way to store a computer's operating system is on the local hard drive, but a sufficiently smart network card can use information provided by DHCP to find an initial boot loader.

All amd64 and modern i386 hardware use Intel's Preboot Execution Environment (PXE, pronounced "pixie"). The DHCP server tells the network card the name of a file and the IP address where the file can be found, and the server fetches the file via TFTP. This file is usually called *pxeboot*, but *pxeboot* files can vary widely among operating systems. A *pxeboot* file for OpenBSD probably won't boot a FreeBSD system, let alone anything from Microsoft. It's specific to each operating system.

Once the computer has loaded *pxeboot*, it goes back to the TFTP server to look for the appropriate kernel. An OpenBSD *pxeboot* looks for a file called *bsd*, assumes that it's a kernel, loads the kernel into memory, and boots it. To install OpenBSD, you'll load the install kernel file *bsd.rd* instead, which you can do automatically.

Diskless Hardware

OpenBSD systems installed over diskless systems must have enough smarts to find their boot loader and operating system over the network or they won't boot. Any machine built in the past several years uses PXE.

You've probably seen a computer try to boot from the network more than once, and for most people, those BIOS messages are just an annoyance that they keep forgetting to disable. For diskless installation, you need to make sure that feature is on.

To enable PXE, boot the hardware and go into the BIOS setup. Somewhere in the BIOS, you should find an option to set the device boot order. If the machine supports PXE, one of those options will be to boot over a network. Enable that option and see if it works. While you're in the BIOS, make a note of the MAC address of your network card. Your DHCP server will need it. If your BIOS uses the Unified Extensible Firmware Interface (UEFI) by default, disable that.

Save your changes and exit. Your hardware should now be prepared. Let's ready the server.

DHCP Server Setup

DHCP is not just a way to hand out IP addresses and network configurations. A DHCP server can tell network-aware phones where to find their configuration, server hardware where to find its operating system, printers where to find their print server, and so on. Diskless installations use DHCP to feed diskless servers the location of the *pxeboot* file.

Per-Host or Per-Network Configuration

DHCP expects to configure hosts either by the network or by the host. When a DHCP server receives a DHCP request, it knows the address of the network that the host is on and the host's MAC address. The DHCP server must decide which configuration to give the host based on this information. This means you can configure your DHCP server so that any host on a given network is told to install OpenBSD, or you can give it the MAC address of the machine you're going to install and tell the DHCP server to start the installation only on that machine.

Because I install machines frequently, I usually set up a small VLAN where any machine plugged onto the network is told to install OpenBSD. That way, workers who plug their laptops into random Ethernet cables in my office get a free operating system upgrade. If you only occasionally install machines, and have control over the DHCP server, it's pretty easy to configure the DHCP server to tell a host with a specific MAC address to install OpenBSD.

The DHCP server needs to tell the client the location of a PXE boot file, which gives the client just enough brains to find a bootable kernel. This is just like the on-disk boot loader, except that the PXE boot file talks to the network. OpenBSD's i386 and amd64 platforms include the file */usr/mdec/pxeboot* for just this purpose.

Give the name of the PXE boot file with the `filename` option, and then use the `next-server` option to specify the IP address of the TFTP server where the client can get the file. This example tells DHCP clients to load the file *pxeboot* from the server at 192.0.2.34:

```
filename "pxeboot";
next-server 192.0.2.34;
```

Place these statements according to whether you have an installation network or your DHCP server is set for a specific MAC address.

Per-Network Configuration

If you want all the hosts on your network to receive the OpenBSD installation PXE boot file, put the `filename` and `next-server` options in the `subnet` stanza, like this:

```
option  domain-name "michaelwlucas.com";
option  domain-name-servers 192.0.2.1;
subnet 192.0.2.0 netmask 255.255.255.0 {
        option routers 192.0.2.1;
        range 192.0.2.10 192.0.2.15;
```

```
        filename "pxeboot";
        next-server 192.0.2.34;
}
```

Any host on this network that makes a DHCP request at boot will learn where to get the PXE boot file.

Per-Machine Configuration

If you've hard-coded a machine's MAC address into your DHCP configuration, as discussed in Chapter 16, you can feed the PXE boot information to that host.

```
subnet 192.0.2.0 netmask 255.255.255.0 {
...
    host installationtarget {
        hardware ethernet 02:03:04:05:06:07;
        filename "pxeboot";
        next-server 192.0.2.34; }
    }
```

Machines on this subnet that make a PXE request at boot will get the location of the PXE boot file only if they have MAC address 02:03:04:05:06:07.

Decide how you want your DHCP server to behave and make similar configuration changes.

Now let's look at the TFTP server.

TFTP Server Setup

The next task is to make the OpenBSD-specific boot files available on your TFTP server. As a minimum, you need the *pxeboot* file and a kernel, but adding a *boot.conf* file will simplify your life.

OpenBSD includes an architecture-specific *pxeboot* file in */usr/mdec/*. If you're installing an i386 machine, grab this file and */bsd.rd* from an existing i386 installation. If you're installing amd64 hardware, get *pxeboot* and */bsd.rd* from an existing amd64 system. Copy them to the TFTP server root directory, and verify that they're world-readable.

pxeboot tells the machine to look for the standard kernel */bsd*, not the installation kernel */bsd.rd*. When *pxeboot* finishes loading, it looks exactly like the standard OpenBSD boot loader. You could interrupt the boot, as described in Chapter 5, and choose a different kernel, but *pxeboot* also recognizes */etc/boot.conf*.

To tell *pxeboot* to load a different kernel, create an *etc* directory in your TFTP server's root directory, and then create the file *boot.conf* inside that. This new *boot.conf* file has exactly the same syntax as */etc/boot.conf*, so you can do a one-line entry like this:

```
boot bsd.rd
```

You can include additional boot options, such as setting a serial console.

Completing Diskless Installation

Once you have DHCP and TFTP, power on the installation target. You should see the network card make a DHCP request, get an IP address, and grab *pxeboot* via TFTP. You should then see the OpenBSD boot loader load the installation *bsd.rd*. Finally, you should get the OpenBSD install script.

If you don't get the installer, take a step back. Does the network card get an address from DHCP? If not, check your wiring and DHCP server configuration. If you get an IP address, but can't fetch *pxeboot*, check that you put the `filename` and `next-server` statements in the correct part of your DHCP configuration, and verify that you don't have a packet filter blocking access to the TFTP server. Try to fetch those files from a different TFTP client to make sure that the TFTP server works. If the installation target partially boots OpenBSD, but doesn't activate the installer, make sure you have an *etc/boot.conf* entry pointing the client at *bsd.rd* rather than *bsd*.

At this point, you should be able to install OpenBSD normally, as described in Chapters 2 and 3. But what if you want to run a full OpenBSD system without a hard drive? That's where diskless operation comes in.

Running Diskless

If you manage many computers, you probably understand that moving parts cause trouble. Spinning hard drives, in particular, are just a very bad idea.

Try this: If you have a roomful of identical machines, try simplifying maintenance by running them without hard drives. Each machine in this group will use a root directory and filesystem mounted via NFS rather than stored locally. You'll still need data storage, but you can use a central high-availability disk array, flash drives, or some other mechanism with better reliability than lowest-common-denominator hard drives.

You can extend the diskless installation process to run OpenBSD in full multiuser mode without a local hard drive. Your server will need three additional services to support fully diskless clients: `rarpd(8)`, `bootparamd(8)`, and NFS. (Only diskless clients need `rarpd` and `bootparamd`.)

Using rarpd(8) for Reverse ARP

In a standard ARP request, a client knows an IP address and wants to get the corresponding MAC address. For reverse ARP, a client knows a MAC address and wants to know the corresponding IP address. OpenBSD needs to get reverse ARP during the diskless boot process, and it uses `rarpd(8)` to provide reverse ARP services to other hosts.

`rarpd` uses */etc/ethers* as a table of Ethernet addresses and hostnames. Each diskless client needs an */etc/ethers* entry much like this:

```
00:50:56:00:01:01       gill.blackhelicopters.org
```

This entry means that the host with MAC address 00:50:56:00:01:01 has the hostname *gill.blackhelicopters.org*. The `rarpd` server must be able to resolve the hostname to an IP address, either in DNS or in */etc/hosts*.

Now decide which network interfaces you want to run `rarpd` on. If your server has only one network interface, that's the one to use. If you have multiple network interfaces, however, it might make sense to listen on only a single interface.

To use a specific interface, use the interface name as a command-line argument; otherwise, use `-a` to listen on all network interfaces. For example, this *rc.conf.local* entry tells `rarpd` to listen on only interface `em0`:

```
rarpd_flags="em0"
```

Start `rarpd` with */etc/rc.d/rarpd*, and go on to `bootparamd`.

Running bootparamd(8)

The boot parameter daemon `bootparamd` tells a diskless OpenBSD machine where to find its root filesystem. When a boot parameter request arrives at the server, `bootparamd` checks the file */etc/bootparams* for a matching configuration and returns that to the client.

Entries in */etc/bootparams* give a hostname, followed by the string `root=`, an NFS server, and the directory where the client's root directory is stored.

```
gill.blackhelicopters.org root=192.0.2.34:/var/diskless/client1
```

In this example, the host *gill.blackhelicopters.org* will use an NFS root directory from a server at 192.0.2.34, in the directory */var/diskless/client1*.

For almost all environments, you can run `bootparamd` without any command-line options. Enable it in *rc.conf.local* like so:

```
bootparamd_flags=""
```

Start `bootparamd`. Now it's time to deal with your NFS server.

Setting Up the NFS Root Directory

A multiuser OpenBSD system needs a userland. Without a local disk, you'll need to create an OpenBSD userland. It is possible to export the NFS server's root directory for use as the diskless client's root directory, but this isn't merely insecure, it's also a good way to damage the NFS server itself. Create a separate userland for your diskless machine.

Exporting the Root Directory

You must export the userland's root directory to the diskless machine. For example, here's an */etc/exports* line that shares the directory */var/diskless/client1* to the IP address 192.0.2.37:

```
/var/diskless/client1 -maproot=root 192.0.2.37
```

Note the `-maproot` option here. The diskless client will expect to be able to write and own files as the root user. This `-maproot` entry maps UID 0

(root) on the client to the root account on the NFS server. You can also set up a separate user for the diskless client's root account, map the client's root account to that new account, and change the ownership of all files in the diskless userland to that root account. As this is your first diskless host, however, we'll start off basic.

Populating the Diskless Userland

The easy way to install a minimal userland is to extract the *etcXX.tgz* and *baseXX.tgz* file sets from your chosen OpenBSD release into the NFS root directory. In the following example, I've copied these file sets into */tmp*, and I'm using them to create a userland in */var/diskless/client1*.

```
# cd /var/diskless/client1
# tar -xzpf /tmp/etc53.tgz
# tar -xzpf /tmp/base53.tgz
```

Note the use of the `-p` flag in the `tar` command, preserving the original permissions on extracted files.

The diskless client also needs device nodes. Go into the new userland's *dev* directory and create them.

```
# cd dev
# ./MAKEDEV all
```

While `bootparamd` told the kernel where to find the root of the filesystem, userland programs expect to read */etc/fstab* for that information. Create an */etc/fstab* file that points the root directory to your NFS share.

```
192.0.2.34:/var/diskless/client1 / nfs rw 0 0
```

You can also add any other NFS-mounted directories you desire here. This should be everything you need.

Power On!

Once you have a basic userland, device nodes, and a filesystem table, you can power on your diskless node, and it should boot. If it doesn't boot to a login prompt, read the console error messages. Usually, they're pretty clear.

Because you've bypassed the OpenBSD installer, there are no root password or user accounts yet. Immediately, log in as root and change the root password, and then set up a regular user account.

For your first diskless setup, once you have a working userland, back it up right away. Even a tar file containing the entire userland will prove useful. You'll muck up the diskless userland more than once as you're trying to get things working exactly as you wish, and being able to blow the entire userland away and restore it from the backup file is invaluable.

Once you have a basic system working, expand it. Add additional file sets as needed, set up more users, add packages, and deploy for your users.

Congratulations, you're now on the cutting edge of OpenBSD users.

USB Installation Media

For many people, burning a CD to install an operating system seems like a waste. They prefer to write an image to a USB flash drive and install from that. OpenBSD doesn't provide such an image, but if you're willing to do some extra work, you can create a bootable USB device that you can use to install on your target hardware.

The official recommendation is to install OpenBSD on the USB device, copy *bsd.rd* and the file sets to that device, and use that to install your new hardware. The OpenBSD installer lets you choose the target hard drive. You select the USB device in the installer, and OpenBSD installs to the USB just as it would any other data-storage device. But how do you install OpenBSD on the USB device without burning a CD in the first place? There are a few ways around this, including a couple of approaches already covered in this chapter.

Using a Virtual Machine

Your first choice is to perform the USB installation in a virtual machine. Many desktop virtual machine software packages let you attach a physical USB port to a guest virtual machine. (OpenBSD's virtualization options are discussed in "Virtualizing OpenBSD" on page 450.)

If you have virtualization software that runs OpenBSD and supports USB, choose this option.

Running a Diskless Installation

Your second choice is to run a diskless installation. Most DHCP servers embedded in cheap home hardware will let you send a filename and a TFTP server address to a client. If yours won't, you can get suitable DHCP servers for any platform. You can find freely available TFTP servers for just about any operating system.

Boot your install target with the USB drive, but load the *bsd.rd* kernel. You now have the OpenBSD installer running on the target system, and an OpenBSD system that fits in your pocket and that you can run almost anywhere. If you're already running OpenBSD on something with the right architecture and a USB socket, it's even easier: You boot the system from the appropriate *bsd.rd*, choose the `disk` option, and point the installer to sets in a local directory.

Converting ISO Images

As a less official method, you can find software to convert ISO images to bootable USB images. I've used Rufus (*http://rufus.akeo.ie/*) on Windows and UNetbootin (*http://unetbootin.sourceforge.net/*) on other Unix-like systems. This approach might work, but it's certainly not OpenBSD-approved.

Customizing OpenBSD Installations

Many of us follow a set of steps when installing a machine. All freshly installed hosts of a specific operating system revision have a common SSH server configuration. My machines all have `tcsh` installed and attach to the central authentication system. You probably have your own list. These tasks can be done by hand after installation, but it's much easier to let OpenBSD do them for you during the installation process.

Installations can be customized by adding files during installation or by running commands after the installation.

Custom File Sets

A custom file set includes files that you want copied to your new installation. I use custom file sets to install the default */etc/sudoers*, a SSH server configuration, my company's default *pf.conf*, and similar files. As I'm the lead sysadmin, I also include dotfiles in my home directory and other personal touches to make my life easier. Some people include several home directories, including *authorized_keys* files for SSH.

Bundle these files together as a *siteXX.tgz* file, which the installer can extract in the root directory of the new installation. (Be sure to replace the *XX* with the OpenBSD version you're installing on; for example, name a *siteXX.tgz* file for OpenBSD 5.4 *site54.tgz*.)

Start by installing an OpenBSD machine of the exact same version and platform that you want to customize. Make your changes and add your files to this system, verify that this template system works exactly as you desire, and then copy the changed files to a tar file.

NOTE *You could make a directory hierarchy and copy the files you want to it, but I find that to be more error-prone. A small virtual machine will let you build a* siteXX.tgz *file more reliably.*

The following example creates a *site54.tar* file containing one file, */etc/ssh/sshd_config*. Note that I start by creating a plain tar file. Since I can't easily add files to a compressed tar file, I'll need to compress the file after it's complete.

```
# cd /
# tar -cf site54.tar etc/ssh/sshd_config
```

Now that I have the initial file, I can add additional files. I've customized a few files on the system, as well as added new ones, all of which I add to the *site54.tar* file. The `-r` flag tells `tar` to add a file to an archive.

```
# tar -rf site54.tar etc/sudoers
# tar -rf site54.tar etc/pf/mgmt-hosts.conf
# tar -rf site54.tar etc/pf.conf
```

Here's how to compress the tar file:

```
# gzip site54.tar
# mv site54.tar.gz site54.tgz
```

I've built my own custom release of OpenBSD, so I have a local FTP server that contains all of the release files. If you're using the official OpenBSD release, but you're installing enough OpenBSD machines to warrant making a *siteXX.tgz* file, you can copy the official release to a local FTP or HTTP mirror. Copy your *siteXX.tgz* file to this directory and update the *index.txt* file.

```
# ls -l > index.txt
```

Now start your installation. Tell the installer to use your local release mirror rather than an official OpenBSD mirror. You should see the following sets:

```
Select sets by entering a set name, a file name pattern or 'all'. De-select
sets by prepending a '-' to the set name, file name pattern or 'all'. Selected
sets are labelled '[X]'.
    [X] bsd           [X] etc54.tgz     [X] xbase54.tgz   [X] xserv54.tgz
    [X] bsd.rd        [X] comp54.tgz    [X] xetc54.tgz    [ ] site54.tgz
    [ ] bsd.mp        [X] man54.tgz     [X] xshare54.tgz
    [X] base54.tgz    [X] game54.tgz    [X] xfont54.tgz
Set name(s)? (or 'abort' or 'done') [done] site52.tgz
```

Your *site54.tgz* file should now be available as a file set. Add it because the installer won't automatically include it. Once the installation finishes, you should find your customized and added files on the new system.

Post-Install Shell Scripts

Some tasks can be accomplished by copying files, but that's annoying. For example, I want the shell `tcsh` installed on all of my OpenBSD servers. I could put all the files in the `tcsh` package, as well as the contents of */var/db/pkg/tcsh*, in *siteXX.tgz*, but I know I'm likely to mess that up somehow. It would be much easier to run `pkg_add tcsh` after the installation, and let OpenBSD do what it's supposed to do. That's where the `install.site` script comes in.

After completing the installation, but before giving you the final command prompt, OpenBSD checks for */install.site*. If this file exists, the installer runs it. The script is run `chroot`ed into the new installed system, so you don't need to worry about changing any paths. The script does need to run on a minimal kernel, however, so it's best to wait for low-level kernel twiddling until the first real boot.

Here's a sample `install.site` script that installs the two packages `tcsh` and `python`:

```
#/bin/sh
export PKG_PATH=ftp://ftp13.usa.openbsd.org/pub/OpenBSD/snapshots/packages/i386/
```

```
pkg_add -v tcsh
pkg_add -v python-2.7.3p1
```

When working with `install.site` scripts, if a package's name could be ambiguous, be sure to give the full package name. There's only one `tcsh` package, but Python comes in several versions. I specify the full package name, rather than using plain `python`.

Also note that while you're running in a `chroot` that contains a full userland, that userland isn't fully initialized. When dropping into the `chroot`, OpenBSD doesn't do a full multiuser startup of that `chroot`. The environment is roughly equivalent to single-user mode. The `install.site` script is not where you initialize your database.

When you have a real userland ready to go, to automatically run commands on the system's first real boot, append the commands to */etc/rc.firsttime*. This file runs once, at the system's first boot after installation, and then deletes itself.

Customizing Upgrades

OpenBSD lets you use custom file sets and shell scripts during binary upgrades. If you have a lot of machines to upgrade, run these to ensure that your systems are as identical after the upgrade as they were before. I highly recommend automating known changes during an upgrade.

The *siteXX.tgz* file works for upgrades exactly as for installations. Put the files you want on this system in *siteXX.tgz*, and the install program should copy those files to the system as it installs the upgraded files. Rather than `install.site`, however, the upgrade software looks for the script `upgrade.site`. Any *install.site* file is ignored during an upgrade, so you can use the same *siteXX.tgz* for upgrades and for new installations.

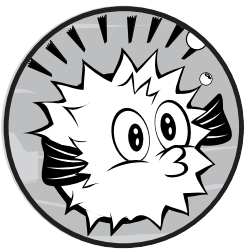
I find the `upgrade.site` script especially useful in conjunction with the *OpenBSD Upgrade Guide* for that release. The *Upgrade Guide* includes tasks that must be performed during an upgrade, many of which are very suitable for scripting. For example, the common tasks of deleting files, programs, and libraries removed from the new OpenBSD release are easily added to `upgrade.site`.

One convenient thing about `upgrade.site` is that you can copy the script to the target machine before running the upgrade. It doesn't need to be part of *siteXX.tgz*. That said, I don't recommend running `pkg_add -u` in `upgrade.site`. While the idea of automatically upgrading all your packages sounds good, remember that you're running on a limited kernel with a less than completely initialized userland. Have your `upgrade.site` script add any commands that need to run on a fully multiuser system to */etc/rc.firsttime*, so that they run when the system boots the first time.

With the hints in this chapter, you can customize OpenBSD any way you need. And with the information throughout this book, you should know where OpenBSD fits into your network. Remember that they really are out to get you, and you'll achieve practical paranoia.

AFTERWORD

Failure's bad enough;
add the human element
and things really suck.



Back around 2000, my employer's main business was designing web applications, but once those applications were built, our clients would turn around and ask, "Where should we host this?" That's where I came in, building and running a small but professional-grade datacenter for custom applications.

As with any new business, our hosting operation needed to make the most of existing resources. Hardware was strictly limited to cast-off equipment from the web developers, and we used only software that was free. The only major expense was a big-name commercial firewall, purchased for marketing rather than technical reasons.

With a whole mess of open source software, we built a reliable network management system that provided our clients with more insight into their equipment than their in-house people could offer. The clients paid for their own hardware, and so had fancy high-end rackmount servers with

their chosen applications, platforms, and operating systems. As the business grew, we upgraded the hardware (it's nice to have disk drives that are less than five years old), but we saw no need to replace the software.

One Monday morning, a customer who had expected to use very little bandwidth found that he had sufficient requests to devour twice the bandwidth we had for the entire datacenter. This affected every customer. If your $9.95 per month web page is slow, you have little to complain about; if your $50,000 per month web application is slow, you pick up the phone and scream until the problem stops.

To make life worse, my grandmother had died only a couple days before. Visitation was on Tuesday, and the funeral was Wednesday morning. I handed the problem to a minion and said, "Here, do something about this." I knew the network could manage bandwidth at many points. The web servers themselves, the load balancer in front of them, the commercial firewall, and even the router claimed to have traffic-management capacity.

Tuesday, after visitation, my cell phone voicemail was full. Our version of Internet Information Server (IIS) could manage bandwidth—in 8MB increments, and only if the content was static HTML and JPEG files. With several web servers behind the load balancer, that fell somewhere between useless and laughable. The load balancer *would* support traffic shaping, if we bought the new feature set. If we plopped down a credit card, we could have that feature set installed by next Sunday. Our big-name commercial firewall also had traffic-shaping features available, *if* we upgraded our service level and paid an additional (and quite hefty) fee for the feature set. That left the router, which I had previously investigated and found would support traffic shaping with only an IOS upgrade.

I was on the phone until midnight Tuesday night, making arrangements to do an emergency router IOS upgrade on Wednesday night. I had planned to go to the funeral Wednesday morning, give a eulogy, go home and take a nap, and arrive at work at midnight ready to rock.

Unfortunately, the funeral was more dramatic than I had expected, and I showed up at work at midnight sleepless, bleary-eyed, and upright only courtesy of the twin blessings of caffeine and adrenaline. In my email, I found a note that several big clients had threatened to leave unless the problems were resolved by Thursday morning. If I hadn't already been stressed out, the prospect of choosing a minion to lay off would have done the trick. (I work hard training my minions, and prefer not to replace them once they are beaten into shape.)

Still, only a simple router flash upgrade and some basic configuration stood between me and relief. What could possibly go wrong?

The upgrade went smoothly, but the router behaved oddly when I enabled traffic shaping. Over the next few hours, I discovered that the router didn't have enough memory to simultaneously support all of our BGP feeds and the traffic-shaping functionality. Worse, it wouldn't accept more memory. At about 6:00 AM, I finally got an admission from the router vendor that it could not help me.

I hung up the phone. The first client who had threatened departure would be checking in at 7:30 AM. I had slept 4 hours of the last 48, and had spent most of that time under fiendish levels of emotional stress. I had already emptied my stash of quarters for the soda machine, and had pillaged my coworkers' desks for more change. The caffeine and adrenaline that had gotten me to the office had long since worn off, and further doses of each merely slowed my collapse. We had support contracts on every piece of equipment, and they were all useless. All the hours of work I had put in, and my team before me, left me with absolutely nothing.

I made myself sit still for two minutes simply focusing on breathing, making my head stop sliding around loose on my shoulders, and ignoring the loud ticking of the clock. What could be done in 90 minutes—no, now only 88?

I really had one only option. If it didn't work, I would either lay off someone or file for unemployment.

At 6:05 AM, I started downloading the OpenBSD install floppy image, and then I grabbed a spare desktop machine, selecting it from among many similar machines by virtue of it being on top of the pile. The next few minutes, I alternated between hitting the few required installation commands and dismantling every unused machine unlucky enough to be in reach to find two decent network cards.

By 6:33 AM, I had two Intel EtherExpress cards in my hands and a virgin OpenBSD system. I logged in long enough to shut down the system so I could wrench the case off, slam the cards into place, and boot again. Even early versions of PF included all sorts of nifty filtering abilities, all of which I ignored in favor of the newly released traffic-shaping functions. By 6:37 AM, I was wheeling a cart with a monitor, keyboard, and my new traffic shaper over to the rack.

Then things got hard. I didn't have a spare switch that could handle our Internet bandwidth. The router rack was jammed to overflowing, leaving me no place to put the new shaper. I lost almost half an hour finding a crossover cable, and when I discovered one, it was only two feet long. The router, of course, was mounted in the top of the rack. About 7:10 AM, I discovered that if I put the desktop PC on end, set it on an empty shipping box, and put the box on the cart, the cable *just* reached the router. I stacked everything so it would reach, and began rewiring the network and reconfiguring subnets.

I vaguely recall my manager coming in about 7:15 AM, asking with taut calmness if he could help. If I remember correctly, as I typed madly at the router console, I said, "Yes. Go away."

At 7:28 AM, we had an OpenBSD traffic shaper between the hosting area and our router. All the client applications were reachable from the Internet. I collapsed in my chair and stared blankly at the wall.

While everything seemed to work, the proof would be in what happened as our offending site started its daily business. I watched with growing tension as that client's network traffic climbed toward the red line that indicated

trouble. The traffic grew to just short of the danger line, and then flatlined. Other clients called, happy that their service was restored to its usual quality. One client complained that his site was still slow, but it turned out that bandwidth problem had masked a problem with his application. The client said that his website now ran even slower than before, to which we offered to provide more bandwidth if they would agree to pay for it.

Shortly afterward, I had two new routers and new DS3s. The racks were again clean. The decrepit desktop machine was replaced by two OpenBSD boxes in a live-failover configuration, protecting our big-name commercial firewall as well as shaping traffic. And I now stock crossover cables in a variety of lengths.

If I had started with OpenBSD, I would have had a much better night.

INDEX

Symbols

* (asterisk), as wildcard, 285
@ symbol, to send messages to another host, 288
\ (backslash), for line continuation, 78, 113
$ (dollar sign), in pathnames, 96
! (exclamation point)
  to escape to command prompt, 43
  as negation symbol, 117–118
    in filter rule, 406
> symbol, for disklabel(8) command prompt, 50
(hash mark), for comments, 33
% (percent sign), for groups in user aliases, 114
/ (root) partition. *See* root (/) partition
~ (tilde), in pathnames, 96
_ (underscore), for unprivileged user names, 103–104

A

a command, 52
abandoned IP addresses, 310
abbreviations, for disk sizes, 52
ABIs (application binary interfaces), 2
abort (fdisk), 131
account information access, controlling, 266
ACPI (Advanced Configuration and Power Interface), 341
acpi0 device, 341
activ method for BSD authentication, 99
active FTP, 437
active partition, marking, 131
address families, in packet filtering, 405
Address Resolution Protocol (ARP), 185
  IPv4 addresses and, 214
address space layout randomization, 174
adduser(8), 87–89
  -batch flag, 89
  configuring default settings, 87–88
  options, 91–92
administrator accounts, creating, 91–92
Advanced Configuration and Power Interface (ACPI), 341
advanced persistent threat (APT), 171
advocacy@OpenBSD.org, 9
afterboot(8) man page, 57
aggressive optimization for PF, 420
aliases, 113–117
  naming conventions, 117
  nesting, 116
-alldirs option, for mount point in partition, 156
ALTQ bandwidth management system, 439
/altroot partition, 73
  backup to, 148
amd64 platform, 16
  boot floppies, FFS support by, 133–134
  floppy image for, 39
  Intel Preboot Execution Environment on, 451
  kernel configuration directory, 361
anchors in PF, 434, 439
  adding rules, 434–435
  conditional filtering, 436
  nested, 436–437
  viewing and flushing, 436
announce@OpenBSD.org, 8
anonymous CVS, 386
antispoofing rule, 416
Apache web server, 227
APIs (application programming interfaces), 2
application binary interfaces (ABIs), 2
application menu, creating in X Windows System, 334
application programming interfaces (APIs), 2
applications. *See also* software
  PF and, 400–401
  preventing coverage by window, 336
applications layer (OSI), 186–187
apropos(1), 5–6
APT (advanced persistent threat), 171
archives, of mailing lists, 10

ARP (Address Resolution Protocol), 185
IPv4 addresses and, 214
arp(8), 214
asking questions, OpenBSD experts reaction to, 11
asterisk (*), as wildcard, 285
asynchronous mounts, in FFS, 136
AT&T, xxxi, xxxii
atexit(), 174
audio, 268
audio device, 351
auth facility, 283
auth-defaults class, 100, 101
authenticating packet filter configuration, 256
authentication methods for user accounts, 99–100
auth-ftp-defaults class, 100
authorized users, repository of, 157
authpf(8), 101
anchors for, 434
authpriv facility, 283
automation
packet filtering tables and, 425–426
of ports, 236
automounter daemon, 256
autonegotiation, in Ethernet network, 215
availability, xxx

B

back channel in FTP, 437
background color of desktop, 335
backslash (\), for line continuation, 78, 113
backup
to */altroot* partition, 148
in daily maintenance, 280–281
of default kernel, 349
of GENERIC kernel, 358
before install, 37
of userland for diskless station, 456
bandwidth management in PF, 439–445
assigning traffic to queues, 444
child queues, 442
for parent queue, 441
queue options, 442–443
base operating system, preparing for your own OpenBSD, 383–384
baseXX.tgz file set, 24, 456
Basic Input/Output System. *See* BIOS (Basic Input/Output System)
beep of computer, 324
Berkeley Internet Name Domain server (BIND), 211
BerliOS, mirrors for, 247
BGP (Border Gateway Protocol), 203
BGP daemon, 257
bgpd(8), 205, 257
bidirectional NAT, 429–432
and packet filter rule order, 430–431
redirection, 431–432
and security, 430
Big Giant Lock method, 18
bigptrmove, 337
binary objects (blobs), 17–18
binary object device drivers, 17
binat-to keyword, 429
BIND (Berkeley Internet Name Domain server), 211
bind command, for mapping keys, 336
binding, key sequence to cwm command, 332
bioctl(8), 160
-d flag, 165
BIOS (Basic Input/Output System)
in boot process, 70
clock, 45
configuration, 38
bios0 device, 341
blanking screen, 324–325
blobs (binary objects), 17–18
block devices, 126–127
block statement, 404
blocks in FFS, 134
number of used, 143
$BLOCKSIZE environment variable, 143–144
bogons, 422–423
bonding, 221
The Book of PF (Hansteen), 256, 395, 448
Boolean sysctls, 346
boot command, 70
boot loader, 69, 70
information on disk devices, 73
making settings permanent, 74–75
prompt, 70
boot media, 22
creating, 38–40
boot CDs, 40
boot floppies, 39–40
boot process, 69–84
from alternate hard disk, 73–74
in alternate kernel, 72–74
configuring VLANs, 224
delaying, 70
Ethernet network configuration at, 219–220
to graphic console, 67
interrupting, 41, 70
for kernel, 74

kernel configuration in, 353
multiuser startup, 79–84
options before completing, 70–71
serial consoles, 75–79
setting sysctls at, 346–348
setting wscons variables, 325
in single-user mode, 71–72
from softraid(4) devices, 166
trunks at, 222
and X Windows System, 330–331
bootable partition, 131
bootparamd(8) daemon, running, 454
bootstrap tools, installing, 372
boot-time securelevel, 178
Border Gateway Protocol (BGP), 203
borders for windows, 336
botnets, 170
Brauer, Henning, xxv–xxvi, 268*n*, 371*n*
bridge(4) interfaces, 400
broadcast address, 191
broadcast protocol, Ethernet as, 213
BSD, xxxi
license, xxxi, xxxii
BSD authentication, 99
/bsd file, 349
bsd file set, 23–24
bsd.mp file set, 23–24
bsd.rd file set, 23–24
BUFCACHEPERCENT value in kernel, 351
bugs
identifying, 3
in releases, 58
BUGS section, in man pages, 7
build files for ports, 238
_build keyword, 265
building
custom kernels, 365–366
troubleshooting errors in, 365–366
programs, virtual terminal SSH connections for, 325
your own OpenBSD
getting source code, 384
preparations for, 383–388
reasons for, 382–383
burncd (Unix), 40
business card attachments, 13

C

C compiler, 24
C++ compiler, 24
canaries, 174
cap_mkdb(8), 95
CARP (Command Address Redundancy Protocol), 316, 317
CAT5 cable, 76
CBQ (class-based queuing), 440
borrow option, 443
ruleset, 443–444
CD drives, emulating floppies, 40
cdemuXX.iso image, 40
cdio(1), 152
cdrecord (Unix), 40
CDs (compact discs), 153
booting from, 38, 40
mounting, 152
obtaining official, 20
cdXX.iso image, 40
Changelogs directory, 20
character devices, 127
chargen function (inetd), 317
check command, 83
chflags(1), 176
child queue, definitions, 442
chio(1) medium changer, 257
chpass(1), 93, 99, 266
chroot, 460
chrooting users, 319–322
class-based queuing (CBQ), 440
borrow option, 443
ruleset, 443–444
cleaning filesystems, 138, 374
client for serial console, 76–77
port, 78–79
clock in BIOS, 45
correcting, 294
clri(8), for dirty filesystem, 138
collision domain, 213
Command Address Redundancy Protocol (CARP), 316, 317
command alias, 115
command prompt. *See* prompt
comments, hash mark (#) for, 33
committers, xxxv
communities in SNMP, 314
compact discs. *See* CDs (compact discs)
compilers, */usr* partition for, 28
compressed tar files, for code snapshots, 384
Computer Science Research Group (CSRG), xxxi
compXX.tgz file set, 24
concatenated disks, 162
Concurrent Versions System (CVS), 385
mirrors, 386
conditional filtering, anchors for, 436
confidentiality, xxx

config(8)
for kernel changes, 348–353
backup of default kernel, 349
changing constants, 352–353
help and list commands, 350–351
for testing custom kernel, 364–365
configuration, testing by rebooting, 57
connectionless protocol, 197
conservative optimization for PF, 419–420
console, 274
configuration with wscons, 324–325
const keyword (PF), for table, 422
content farms, bandwidth control machines for, 440
contributors to OpenBSD, xxxiv
converting ISO images, 456
cooked device node, 126
Coordinated Universal Time (UTC), 45
coordinator for OpenBSD, xxxv
copycenter, xxxii
copying
disk images to disk, 40
files to other servers, 281
copyleft, xxxii
core programs, 24
coredumpsize variable, 96
country code, for USB keyboards, 66
cp(1), copying files with, 145
cpio(1), copying files with, 145
cputime variable, 96
cron facility, 283
cron(8), 109
cross-compiling, 383
crypto method for BSD authentication, 99
cryptography, OpenBSD support for, 10
csh(1), system-wide defaults for, 257
CSRG (Computer Science Research Group), xxxi
CTRL-ALT-DEL, effect of, 348
current resource limit, specifying, 97
-current version of OpenBSD, 368–369
building, 392–393
source code for, 384
updating to, 387–388
cursor, controlling with keyboard, 335
custom kernels, 355–366
building, 365–366
cautions, 355–358
configuration, 359–365
device drivers, 359
keywords, 360
pseudo-devices, 359–360
configuration file, 362–364
identifying running, 366
installing, 366
preparations for, 358
problems building, 357
problems running, 358
reasons for, 356–357
removing devices, 363
removing options, 362–363
stripping down, 363
testing, 364–365
CVS (Concurrent Versions System), 385
mirrors, 386
CVS directory, 237
cwm(1) window manager, 330, 331–337
configuration file loss, 332
configuring, 331–332
modifier keys, 331–332
creating windows, 332–333
decorating, 335–336
exiting, 333
locking screen, 333–334
resizing terminal window, 333
.cwmrc file, 331
mapping keys in, 336
cylinders, 31

D

d command, 51
daemon
checking for running, 83
instructed to reread configuration file, 83
unprivileged account for each, 103
daemon facility, 283
DaemonForums, 8
daily maintenance, 278–281
daily(8), 278
damaged filesystem, recovering, 139
data connection for FTP, 437
data integrity, synchronous mounts for, 136
datalink layer (OSI), 185, 187
datasize variable, 96
date, setting, 60–61
date(1), 60, 61
DB9-to-RJ45 converters, 76
dd(1), 39–40
ddb.console sysctl, 348
ddb.panic sysctl, 348
de Raadt, Theo, xxxiii, xxxv, xxxvii
decrypted partition
automatic, 168
unmounting, 167
default accept, vs. default deny, 399
default answers, for installer, 42
default BSD pager, 5

default gateway, 64
_default keyword, 264–265
default login class, for user, 87
default partitioning, by installer, 26
default permit or default deny, 404
default route, 203
  adding to routing table, 207
  on Ethernet, configuring, 219
default screensaver in cwm, 334
default search domains, 210–211
default shell for user, 87
default user class, 94
  definition, 94–95
Defaults statement, 117
delete command (pfctl), 425
deleting
  partitions, 51
  routes, 207
  softraid(4) devices, 165
  user accounts, 92
dependencies
  for packages, 232, 234–235, 381
  ports and, 241, 250–251
DESCRIPTION section, in man pages, 7
desktop OpenBSD, 323–337
  background color, 335
  console configuration with wscons, 324–325
  cwm(1) window manager for, 331–337
  tmux for virtual terminals, 325–329
  X graphical interface setup, 330–331
$DESTDIR environment variable, 390, 391
destination address, in filter rule, 406
destination port, in filter rule, 408–409
detaching vnode devices from images, 154
/dev/console file, 274
developers' logs, 2
device drivers
  attachment to hardware, 341
  binary object, 17
  custom kernel configuration for, 359
  enabling, 350, 352
  finding, 352
  for hardware sensors, 297–298
  kernel and, 340, 349
  minimizing number in custom kernel, 363
  in OpenBSD, 41
  for physical sensors, 297–298
device names
  device attachment vs., 127–128
  for floppy drives, 39–40
  for hard drives, 73
device nodes, 126–128
df(1), 142
dhclient(8), 219
DHCP, 212–213
  getting IPv4 address from, 219
  server setup for diskless install, 452–453
  static IP address and, 42–43, 309
dhcpd (DHCP daemon), 307–310
  enabling, 309
dial-up modem, 270
Diffie-Hellman cryptography, 268
directories
  locking users in, 319–320
  for new releases, 390
  number of used filesystem blocks, 143
  for tftpd, 310–311
dirty filesystems, 138
discard function (inetd), 317
disk drives. *See also* hard drives
  CD drives emulating floppies, 40
  changing basic parameters, 54
  device names, 32
  mounting, in single-user mode, 71–72
  setup when installing OpenBSD, 46–47
    custom layout, 49–54
disk images, attaching vnode devices to, 154
disklabel partitions, 31, 50
  creating, 51–53, 132
  for softraid device, 162
disklabel unique identifier (DUID), 33
  and /etc/fstab filesystem table, 128–129
disklabel(8), 25
  command prompt, 50
  expert mode for, 55
  help for, 55
disklabels, 31–34
  advanced commands, 54–55
  backing up and restoring, 133
  creating, 144–145
  erasing, 51
  printing, 50, 53
  viewing, 50–51, 132
  writing new, 53–54
  writing to disk, 53–54
diskless installation, 450–454
  DHCP server setup, 452–453
  power for, 456
  running, 454–456
  TFTP server setup, 453
display. *See* screen
display.kbdact variable, 325
display.msact variable, 325
display.outact variable, 325

display.screen_off variable, 325
Distance Vector Multicast Routing Protocol (DVMRP), 258
$DISTDIR variable, 244
distfiles directory, 20
distfiles, for ports, 244
divert-to keyword (PF), 433
dmassage package, 343
dmesg(8), 340
DMZ, hosts in, 204
DNS (Domain Name Service) servers, 65
DNS queries, 399
DNS resolution, 210–213
  /etc/hosts file, 212
  resolver vs. dynamic configuration, 212–213
DNS spoofing attacks, 115
doc directory, 21
documentation, xxxvi–xxxvii
  distribution set for, 23
  man pages, 3–7
  not provided by vendors, 17
dollar sign ($), in pathnames, 96
domain, 210
Domain Name Service (DNS) servers, 65
"Don't Track Access Time" mounts, in FFS, 137
du(1), 143
dual-stacked setup, 188
DUID (disklabel unique identifier), 33
  and /etc/fstab filesystem table, 128–129
dump(8), 128, 139
  avoiding for NFS mount, 160
  backup, 258
  copying files with, 145
DVMRP (Distance Vector Multicast Routing Protocol), 258
dvmrpd(8), 258
Dvorak layout, 66
dynamic clients, dhcpd for configuring, 307
dynamic configuration, vs. resolver, 212–213
dynamic network configuration, 64

E

e command, 54
echo function (inetd), 317
ECN (Explicit Congestion Notification), 347, 443
$EDITOR environment variable, 110, 122
EISA hardware, 16
email
  attachments, 13
  for help request, 12–13
  maintenance tasks results to local root account, 65
  responding to, 14
  sending, 13–14
email software, configuration files for, 263
embedded systems, and sylogd(8), 289
emergency root partition, 148
emulated CPUs, 19
enc0 (encapsulating interface), 63, 216
encrypt(1), 91
encrypted partitions, 166–168
  automatic decryption, 168
encryption algorithm, for user passwords, 88
Enhanced Small Device Interface (ESDI), 33
environment variables
  in */etc/login.conf* file, 97
  and sudo(8), 119–120
erasing. *See also* deleting
  disklabels, 51
error messages
  mmap: Cannot allocate memory, 149
  NFS-related, 155
  from snmpd, 315
errors, from custom kernel, 364
ESDI (Enhanced Small Device Interface), 33
ESXi, 19
/etc/adduser.conf file, 256
/etc/aliases file, 278
/etc/amd file, 256
/etc/authpf directory, 256
/etc/bgpd.conf file, 257
/etc/boot.conf file, 74–75, 257
/etc/bootparams file, 455
/etc/changelist file, 257, 280
/etc/chio.conf file, 257
*/etc/csh.** files, 257
/etc/daily file, VERBOSESTATUS, 281
/etc/daily.local file, 148, 257, 278
/etc/dhclient.conf file, 257
/etc/dhcpd.conf file, 257, 308
/etc directory, 255
  files in, 256–276
  merging changes, 393
  across Unix variants, 256
  updating, 375–380
    sysmerge(8) to compare files, 376–378
/etc/disklabels/ directory, 257–258

/etc/disktab file, 258
/etc/dumpdates file, 258
/etc/dvmrpd.conf file, 258
/etc/ethers file, 454
/etc/exports file, 156, 258, 455
/etc/fastboot script, 82
/etc/fbtab file, 258
/etc/firmware file, 258–259
/etc/fonts/ directory, 259
/etc/fstab file, 135, 149–150, 259
    CD and FAT flash drive entry, 153
    for mounting NFS share, 160
/etc/fstab filesystem table, and DUID for disk, 128–129
/etc/ftpchroot file, 259
/etc/ftpusers file, 259
/etc/gettytab file, 259
/etc/group file, 107, 260
    editing, 93–94
/etc/hostapd.conf file, 260
/etc/hostname. files, 260
/etc/hostname.if file, `route` statement in, 207
/etc/hostname.interfacename file, 219
/etc/hosts file, 212, 260
/etc/hosts.equiv file, 260
/etc/hosts.lpd file, 260–261
/etc/hotplug/ file, 261
/etc/ifstated.conf file, 261
/etc/iked/ file, 261
/etc/iked.conf file, 261
/etc/inetd.conf file, 261, 317
/etc/ipsec.conf file, 261
/etc/isakmpd file, 261
/etc/kbdtype file, 66, 261
/etc/kerberosV/ directory, 262
/etc/ksh.kshrc file, 262
/etc/ldap/ file, 262
/etc/ldapd.conf file, 262
/etc/localtime file, 59, 262
/etc/locate.rc file, 262
/etc/login.conf file, 94, 262
    changing, 95
    environment variables, 97
    legal values for variables, 95–96
    sample entries from default, 100
/etc/lynx.cfg file, 262
/etc/magic file, 262
/etc/mail/ directory, 263
/etc/mail/aliases file, 65, 263
/etc/mailer.conf file, 263
/etc/mail.rc file, 263
/etc/man.conf file, 264–265
/etc/master.passwd file, 88, 90
    editing, 93–94, 266
    fields, 267–268
/etc/mixerctl.conf file, 268
/etc/mk.conf file, 238, 268
/etc/moduli file, 268
/etc/monthly file, 268
/etc/monthly.local file, 268
/etc/motd file, 269
/etc/mrouted.conf file, 269
/etc/mtree/ directory, 269
/etc/mygate script, 269
/etc/myname file, 61, 269
/etc/named.conf file, 121–122
/etc/netstart script, 62, 81, 219, 269
/etc/networks file, 269
/etc/newsyslog.conf file, 269, 290–292
/etc/nginx/ file, 269–270
/etc/nsd.conf file, 270
/etc/ntpd.conf file, 60, 270, 294, 295
/etc/ospf6d.conf file, 270
/etc/ospfd.conf file, 270
/etc/passwd file, editing, 93–94
/etc/pf.conf file, 270, 397, 401, 422
    anchor rules in, 435
    `anchor` setting, 402
    and FTP proxy, 438–439
    options, 402
/etc/pf.os file, 270
/etc/ppp/ file, 270
/etc/printcap file, 270, 306
/etc/protocols file, 186, 270
/etc/pwd.db file, 266
/etc/raddb/servers file, 101
/etc/rbootd.conf file, 271
/etc/rc script, 59, 79, 80–82, 271
/etc/rc.conf file, 59
    OpenBSD defaults in, 59
/etc/rc.conf script, 80
/etc/rc.conf.local script, 59, 81, 296
    deactivating functions in, 80
    to disable `PF`, 397
    to disable `sshd`, 318
    enabling `dhcpd`, 309
    `ftp-proxy` enabled in, 438
    `snmpd` to enable, 315
/etc/rc.d directory, 82
/etc/rc.firsttime script, 82
/etc/rc.local script, 81
/etc/rc.securelevel script, 81, 178
/etc/rc.shutdown script, 82
/etc/relayd.conf file, 271
/etc/remote file, 76, 78–79, 271
/etc/resolv.conf file, 210–212, 271
/etc/resolv.conf.tail file, 213, 271
/etc/ripd.conf file, 271
/etc/rmt file, 271
/etc/rpc file, 272

/etc/sasyncd.conf file, 272
/etc/sensorsd.conf file, 272, 298–300
/etc/services file, 199, 272, 317, 408
/etc/shells file, 88, 272
/etc/skel/ file, 272
/etc/sliphome/ file, 272
/etc/snmpd.conf file, 273, 314–315
/etc/spwd.db file, 266
/etc/ssh/ file, 273
/etc/ssh/sshd_config file, 318–319
/etc/ssl/ file, 273
/etc/sudoers file, 110, 111–113, 115–116, 273
  aliases, 113–117
  multiple entries in one field, 112
  running commands as non-root users, 113
/etc/sysctl.conf file, 63, 178, 273, 344, 346–348
  `machdep.allowaperture=2` sysctl in, 330
/etc/syslog.conf file, 273, 284–287
/etc/systrace/ directory, 273
/etc/termcap file, 274
/etc/tmux.conf file, 329
/etc/ttys file, 274–276
/etc/weekly.local script, 276
/etc/wsconsctl.conf file, 276
/etc/X11 directory, 276
etcXX.tgz file set, 24, 456
/etc/ypldap.conf file, 276
Ethernet, 209, 213–215
  configuring, 215–220
    default routes, 219
    dynamic network, 219
    network at boot, 219–220
  multiple IP addresses on one card, 218
  speed and duplex, 215
Ethernet cards, configuration file, 63
Ethernet interfaces, 62–64
ex2fs filesystem, 152
ex3fs filesystem, 152
exclamation point (`!`)
  to escape to command prompt, 43
  as negation symbol, 117–118
    in filter rule, 406
exclusions to packet filtering lists, 415
execution forbidden mounts, 137–138
`exit` command (`config`), 352
`exit` command (`fdisk`), 131
exiting, `cwm`, 333
expert mode, for `disklabel`, 55
Explicit Congestion Notification (ECN), 347, 443
exporting filesystems, 155
exports, NFS client mounting of, 155

F

facilities for system logs, 283–284
  combining, 285
failover, 221
fallback mirrors, 248–249
`family` keyword, 212
Fast File System (FFS), 29, 133–140
  blocks, fragments, and inodes, 134
  filesystems
    creating, 134–135
    integrity, 138–140
  mount options, 135–138
  versions, 133–134
FAT filesystems, 150–151
`fdisk(8)`, 30
  exiting, 131
  and MBR partitions, 129–131, 144
`feh`, 335–336
FFS. *See* Fast File System (FFS)
FIFO (first-in, first-out) queuing, 440
file flags, 175–177
file sets, 23–25
  custom, 458
  selecting when installing OpenBSD, 47–49
  for upgrade, 375
File Transport Protocol. *See* FTP (File Transport Protocol)
files
  assigning ownership of, 108
  copying to other servers, 281
  identifying origin of, 232–234
  logging to, 287
  removing during upgrade, 372
`filesize` variable, 96
filesystems
  adding new, 146
  cleaning, 138, 374
  exporting, 155, 156–157
  foreign, 150–152
  impact of block size on partition size, 53
  integrity checks, 71–72, 281
    partition size and, 26
  integrity in FFS, 138–140
  memory, 148–150
  mounting at nonstandard locations, 141
  mounting for upgrade, 376
  mounting images, 153–154
  mounting remote, 159
  mounting standard, 141
  partition, 29

recovering damaged, 139
type on partition, 34
filtering, tcpdump, 447–448
finding. *See* search
fingerprint scanners, 100
firewall, 204, 396–397
avoiding install of unneeded file sets, 48
dhcpd and, 309–310
NAT and, 426
firmware, 17–18, 259
first-in, first-out (FIFO) queuing, 440
flag command (fdisk), 131
flash drives, 153
$FLAVOR environment variable, 250
floppy disk
boot loader information on drive, 73
booting installer from, 22, 38–40
floppyBXX.fs image, 39
floppyCXX.fs image, 39
floppyXX.fs image, 39
flush command (pfctl), 425
force-starting software, 83–84
foreign filesystems, 150–152
fragmentation behavior of filesystem, 34
fragmented packets, 399
frags limit and, 418
fragments in FFS, 134
frame in datalink layer, 187
free lines, in kernel, 351
free use of OpenBSD, xxxvii–xxxviii
fsck(8), 71–72, 374
avoiding for NFS mount, 160
for dirty filesystem, 138
running, 139
trusting, 139–140
fsdb(8), for dirty filesystem, 138
fstat, 202
fstype, 34
FTP (File Transport Protocol)
password source for connections, 101*n*
pf(4) and, 437–439
proxy, PF configuration and, 438–439
ftp facility, 283
FTP server, installing OpenBSD from, 23, 48
ftplist file, 21
ftp-proxy(8), 438–439
anchors for, 434
full-duplex connection, 215
fvwm(1), 331
fw_update script, 259

G

gameXX.tgz file set, 24
gateway, default, 64
GENERIC kernel, 356
backup of, 358
configuration file, 361
configuring, 360–361
geometry of disks, 31
getty(8), 79, 80
GID (group unique number), 107
gigabytes, for displaying partition size, 51
global settings in tmux, 329
Gnome, 331
Gnu C Compiler Project, 247
GNU mirror site, 247
graphic console, 44
booting to, 67
group unique number (GID), 107
groups, 106–109
creating, 107–108
unprivileged user accounts and group permissions, 108–109
in user aliases, 114
user assignment to, 88
in batch mode, 90
guard pages, 174

H

hacker, definition of, 172
half-duplex connection, 215
Hansteen, Peter, *The Book of PF*, 256, 395, 448
hard drives, 18–19. *See also* disk drives
adding new, 144–146
booting from alternate, 73–74
finding for booting, 73
multiple, 29–30
partitioning additional, 54
hardened host, 398
hardware, 16–19
connection to kernel, 342
device driver attachment to, 341
diskless, for OpenBSD, 451
dmassage to view installed devices, 343
setup for installing OpenBSD, 38
hardware sensors, 296–301
configuring, 298–301
device drivers for, 297–298
time, 295
triggering action, 300–301
hardware serial console, 75
hash mark (#), for comments, 33

HDLC (High-Level Data Link Control), 185
heads on disk drives, 31
help
  creating good request, 12–13
  for `disklabel` prompt, 55
`help` command (`config`), 350–351
`help` function, 70
hiding `cwm` windows, 333
`high-latency` optimization for PF, 420
High-Level Data Link Control (HDLC), 185
  mapping keys in, 336
home directories
  configuring for user, 87
  macros to represent, 319–320
  for unprivileged users, 102
/home partition, 29, 87
  creating, 53
$HOME/.cwmrc file, 331
$HOME/.tmux.conf file, 329
$HOME/.xsession file, 331
host access point daemon, 260
host aliases, 115
host MIBs, 313
`hostapd(8)`, 260
hosting operation, 461
`hostname`, 62
hostname.if file, 219
hostnames, 210
  setting, 61–62
  of system, 42
hosts file, 210
HTML, avoiding for email help request, 13
HTTP protocol, 199
  installation, 22–23
hubs, in Ethernet network, 213
Hurricane Electric, IPv6 tunnel service, 224
`hushlogin` variable, 97
`hw.allowpowerdown` sysctl, 179
`hw.ncpufound` sysctl, 345

I

i386 platform, 16
  boot floppies, FFS support by, 133–134
  floppy images for, 39
  hard drive size limitations, 26–27
  Intel Preboot Execution Environment on, 451
ICMP (Internet Control Message Protocol), 186, 196
  redirects, sysctl to control, 347
  states, 413
`id(1)`, 107
IDE drives, 29
`ifconfig(8)`, 62, 63, 191, 194, 216–218, 219–220
  `delete` option, 217
  to display VLAN interface, 224
`ifstated(8)` (interface state daemon), 261
`ifTable` (interface table), 313
`ignorenologin` variable, 97
illegal packets, 415
`in` keyword, for direction in packet filtering, 404–405
`include` statement
  in kernel configuration, 360
  in *pf.conf*, 445
incoming connections, restricting, 305
INDEX file, 236
  building database of, 240
index nodes. *See* inodes (index nodes)
indexing OpenBSD release, 392
`inet`, 219–220
`inetd(8)` (small-server handler), 261, 304–305
`init(8)`, 59, 70
inodes (index nodes), 134
  vs. vnodes, 150–151
install document, in OpenBSD release, 15
installation preparations, 15–35
  customizing, 458–460
  disklabels, 31–34
  file sets, 23–25
  getting OpenBSD, 19–23
  multiple hard drives, 29–30
  OpenBSD hardware, 16–19
  partition filesystems, 29
  partitions, 25–29, 30–31
installing
  custom kernel, 366
  multiple operating systems on computer, 37
  packages, 230–232
  ports and source code, 66
installing OpenBSD. *See also* installation preparations; setup after install
  BIOS configuration, 38
  boot media creation, 38–40
    boot CDs, 40
    boot floppies, 39–40

disk drive
custom layout, 49–54
setup, 46–47
file sets selection, 47–49
hardware setup, 38
multiple network cards and, 43–44
running installation program, 41–43
setting time zone, 45–46
setting up services, 44
installXX.iso image, 40
integrity, xxx
of packages, 229
Intelligent Platform Management Interface (IPMI), 297
interface groups, 401–402
interface main address, in filter rule, 407–408
interface state daemon (`ifstated(8)`), 261
interfaces
dynamic configuration, 220
setting PF to not manage, 420
interim releases of OpenBSD, 369
Internet, finding packages on, 230
Internet connection, 183. *See also* TCP/IP
Internet Control Message Protocol (ICMP), 186, 196
redirects, sysctl to control, 347
states, 413
Internet downloads of OpenBSD, 20
Internet Protocol (IP), 185
Internet searches, on OpenBSD crypto hardware, 11
Internet small service listener (`inetd(8)`), 261
interrupt request (IRQ), 349
interrupting boot process, 41
IP addresses, 185, 189–192
abandoned, 310
adding, 217
multiple, on one Ethernet card, 218
private NAT, 426
removing, 217
static, 63–64
table in *pf.conf* file, 403
IP aliases, 218
IP routing, 202–207
deleting routes, 207
IPv4, 203–204
route flags, 206–207
`route(8)` for managing, 204–207
`ipcalc` package, 191
IPMI (Intelligent Platform Management Interface), 297
`ipmi(4)` driver, 349
IPsec standard for VPNs, 261
IPv4 addresses
and ARP, 214
netmask calculation, 190–191
network stacks, 188
pitfalls, 192
search for records, 212
special, 192
static, 63
unusable, 191
viewing, 191
IPv4 packets, sysctl to control forwarding, 347
IPv6 addresses, 43, 192–196
assigning, 195–196
format for, 63
NAT and, 426
and neighbor discovery, 214
network stacks, 188
special, 194–195
subnets, 194
over tunnels, 224
IPv6 packets, forwarding, sysctl to control, 347
IRQ (interrupt request), 349
ISA hardware, 16, 349
ISO 8601 restricted time format, 291–292
ISO images, converting, 456
ISO-9660 filesystem, 152
`istatus` keyword, to ignore sensor, 300

J

job control, xxxi

K

`kbd(8)`, 66
KDE, 331
Kerberos, 157
`kern` facility, 283
kernel, 23, 70. *See also* custom kernels; sysctls (system controls)
basics, 340–343
boot-time configuration, 353
booting, 74
booting alternate, 72
code snapshot for, 384
`config(8)` for changing, 348–353
backup of default kernel, 349
changing constants, 352–353
`help` and `list` commands, 350–351
device drivers, 349
enabling, 350, 352
finding, 352

kernel (*continued*)
GENERIC, 356
backup, 358
identifying running, 366
messages to userland, 340
modules, 179
startup messages, 340–341
upgrading, 388
`kern.hostname` sysctl, 346
`kern.maxproc` sysctl, 346
`kern.ostype` sysctl, 344, 346
`kern.osversion` sysctl, 344
`kern.version` sysctl, 344
keyboard
mapping, 66
modifer keys in `cwm` window manager, 331–332
unmapping and remapping, 336
for X Windows navigation, 335
keyboard-video-mouse (KVM) system, 75*n*
`keyboard.type` variable, 324
keywords
in custom kernel configuration, 360
for man page searches, 5–6
`kill` command (`pfctl`), 425
`kill -session` command (`tmux`), 328
Kozierok, Charles M., *The TCP/IP Guide*, 184, 397
`krb5` method for BSD authentication, 99
`krb5-or-pwd` method for BSD authentication, 99
KVM (keyboard-vide-mouse) system, 75*n*
KVM hypervisor (Linux), 19
KVM virtualization, 450

L

LACP (Link Aggregation Control Protocol), 221
`lchpass` method for BSD authentication, 99
LDAP (Lightweight Directory Access Protocol) daemon, 157, 262
integration, 100
`LD_LIBRARY_PATH` environment variable, 119
leases in DHCP, 307
least privilege approach, 86
libraries, */usr* partition for, 28
Lightweight Discovery Access Protocol (LDAP) daemon, 157, 262
Link Aggregation Control Protocol (LACP), 221
link aggregation protocols, 221
link local addresses, for IPv6, 195
Linux, KVM hypervisor, 19
`list` command (`config`), 350–351
lists, packet filtering with, 413–414
exclusions and negations, 415
`lladdr` (link local address), 216
`lo0` (loopback) interface, 63, 216
load balancer, 271, 462
local distfile mirrors, 246–249
local installation server, install from, 23
`local0` facility, 283
`localcipher` password control, 98
localhost
for IPv4, 192
for IPv6, 195
locking users, in directory, 319–320
log files
maintenance, 289–294
adding PID file, 293
monitoring, 293
newsyslog.conf fields, 290–292
signal name, 293–294
for PF, 446–448
reading, 447
log rotation, 289–290
log sockets, 288
logging daemon (`syslogd(8)`), 273
customizing, 288
logging host, 288
logical interfaces, in OpenBSD, 216
logical port, 198
login classes, 94–101
definitions, 94–95
for RADIUS authentication, 100–101
for user, 89
`login-backoff` password control, 98
`login_radius(8)`, 101
logins
default class for user, 87
to serial consoles, 79
logs
monitoring, 293
for system maintenance, 282–289
actions, 287–288
customizing `syslogd`, 288
and embedded systems, 289
facilities, 283–284
priority, 284
sorting messages with `syslogd(8)`, 284–287
of TFTP transfers, 311
`lookup` keyword, for DNS resolution, 211
loopback address, 206
lost+found directory, 139
`lpd` (printer daemon), 306–307
`lpr` facility, 283

ls
-la, 108
-lo, 176
lynx(1) text-mode web browser, 262

M

m command, 55
MAC (Media Access Control) addresses, 185, 213
for DHCP server client identity, 307
machdep.allowaperture sysctl, 179, 330, 348
machdep.kbdreset sysctl, 179, 348
machine diskinfo command, 73
machine keyword, in kernel configuration, 360
machine-dependent kernel configuration files, 361
machine-independent kernel configuration files, 360–361
macppc (PowerPC-based Macintosh computers), 16
macros, packet filtering with, 414–415
magic number, 262
mail aliases, setup after install, 65
mail facility, 283
mail server program, 263
mailing lists on OpenBSD, 8–10, 11–14
archives of, 10
read-only, 9–10
mailq, 263
mainbus0, 341
Maint (maintainer), for software, 240
maintenance tasks, emailing results to local root account, 65
make build stage, 390
in port build, 246
make checksum stage in port build, 244–245
make clean stage in port build, 246
make configure stage in port build, 245
make extract stage in port build, 245
make fake stage in port build, 246
make fetch stage in port build, 244
make install stage, 241
in port build, 246
make package stage in port build, 246
make patch stage in port build, 245
make prepare stage in port build, 245
make print-index, 239
make update command, in port, 393
make(1) program, 226
configuring, 268
makefile, 226, 236–237
Makefile.inc, 238
makeoptions keyword, in kernel configuration, 360
makewhatis(8), 264
making software, 226
malloc(), 174
malware, 400
man (manual) pages, xxxvi–xxxvii, 3–7, 264–265
adding to directories, 264–265
contents, 6–7
defining sections, 265
for disklabel(8), 55
displaying, 265
finding, 5–6
moving through, 5
overlapping names, 6
search for cryptography, 10–11
viewing, 4–5
on web, 7
Management Information Base. *See* MIBs (Management Information Base)
management-addresses file, 445
manual pages. *See* man (manual) pages
manXX.tgz file set, 24
mapping
filesystems to mount points, 128
keyboard, 336
mark facility, 283
marker in logs, 286
MASTER_SITE_BERIOS variable, 247
MASTER_SITE_OVERRIDE variable, 249
Match keyword, 320
match keyword, for bandwidth management, 444–445
maximizing cwm windows, 333
maxproc variable, 96
maxusers keyword, in kernel configuration, 360
MBR partitions, 30
creating, 46, 130–131, 144
and fdisk(8), 129–131
for softraid device, 162
viewing, 130
mbrowse package, 313
Media Access Control (MAC) addresses, 185, 213
for DHCP server client identity, 307
megabytes, for displaying partition size, 50
memory (RAM), 18
interface use by, 317
log messages to, 289
protection, 172–175

memory filesystems (MFS), 148–150
mounting at boot, 149–150
partitions, creating, 149
memorylocked variable, 96
memoryuse variable, 96
merging */etc* file changes during upgrade, 379, 393
message of the day (MOTD), 269
messages
displaying for packages, 234
at kernel startup, 340–341
from kernel to userland, 340
from upgraded packages, 381–382
metadata, 134
from softraid, 166
Meyer, Scott, 187
MFS. *See* memory filesystems (MFS)
MIBs (Management Information Base)
PF SNMP, 316
for SNMP, 312–313
sysctl, 343–344
Microsoft systems
Burn to Disc command, 40
floppy creation on, 40
NTFS partitions, 150
virtualization, 450
minpasswordlen password control, 98
mirroring (RAID-1), 161
mirrors
CVS, 386
fallback, 248–249
local distfile, 246–249
of OpenBSD website, 8
preferred collection, 247–248
primary, 249
site layout for obtaining OpenBSD copy, 20–21
misc@OpenBSD.org, 9, 13–14
help for building custom kernel, 357
mixectl(8), 268
mkhybrid(8), 152
monthly maintenance, 282
more(1), 5
MOTD (message of the day), 269
mount command, 154
mount point
mapping filesystems to, 128–129
for partition, 52
mount(8), 140
mount_cd9660(8), 152
mountd startup script, reload argument, 157
mountd(8) daemon, 155
mounted files, listing all, 140
mount_ext2fs(8), 152
mounting
disks
in FFS, 135–138
in single-user mode, 71–72
filesystem images, 153–154
filesystems, for upgrade, 376
filesystems at nonstandard locations, 141
memory filesystems, at boot, 149–150
with options, 142
partitions, 140–142
standard filesystems, 141
mount_mfs(8), 149
mount_msdos(8), 151
mount_ntfs(8), 151
mounts, stackable, 146
mouse, emulating three-button in X, 331
moving, partitions, 145. *See also* navigation
mrouted(8), 269
MS-DOS filesystem, 151
multicast routing, 269
multipackages, 252
multiple hard drives, 29–30
multiple network cards, installing OpenBSD and, 43–44
multiprocessor kernel, 72
multiprocessor support, 18
multiuser startup, 79–84
Mutt mail client, noatime mount option and, 137
mv(1), copying files with, 145

N

NAME section, in man pages, 7
name service servers, 65, 211
named user account, 102
names
for aliases, 117
for default kernel backup, 349
for disk device nodes, 126
for groups, 107
for man pages, overlapping, 6
for user accounts, 92
for windows
in cwm, 333
in tmux, 327
NAT (network address translation), 396
bidirectional, 429–432
redirection, 431–432
and rule order, 430–431
and security, 430
configuring, 427
how it works, 427–428

multiple addresses and interface groups, 432
multiple or specific public addresses, 428–429
packet filtering with, 426–433
port manipulation and ranges, 432–433
transparent interception, 433
nat-to keyword, 427
navigation, through man pages, 5
ND (Neighbor Discovery), 185
IPv6 addresses and, 214
ndp(8), 214
negation symbol, exclamation point (!) as, 117–118
in filter rule, 406
negations in packet filtering lists, 415
Neighbor Discovery (ND), 185
IPv6 addresses and, 214
nested anchors in PF, 436–437
nesting aliases, 116
net.inet6.icmp6.rediraccept sysctl, 347
net.inet6.ip6.accept_rtadv sysctl, 347
net.inet6.ip6.forwarding sysctl, 347
net.inet.icmp.rediraccept sysctl, 347
net.inet.ip.forwarding sysctl, 345, 347
net.inet.ip.sourceroute sysctl, 179
net.inet.tcp.always_keepalive sysctl, 347
netmask, 43
for alias addresses, 218
for IP addresses, 189–190
net-snmp package of command-line tools, 313
netstat, for determining open TCP ports, 200–202
network adapter teaming, 221
network address, 191
network address translation. *See* NAT (network address translation)
network cards, multiple, installing OpenBSD and, 43–44
network connection, 209–224
DNS resolution, 210–213
Ethernet, 213–215
trunking, 221–222
upgrading over, 374–375
VLANs, 223–224
network devices, gathering information on, 312
Network File System. *See* NFS (Network File System)
network interfaces
interrupting installer to identifying, 43
list of recognized, 62
recognition during install, 42
network layers (OSI), 184–187
applications layer, 186–187
datalink layer, 185, 187
network layer, 185–186, 187
physical layer, 184, 187
transport layer, 186, 187
network protocol, for packet filtering, 405–406
network request, data transmission for, 187–188
network servers, 303–322
DHCP daemon (dhcpd), 307–310
printer daemon (lpd), 306–307
small-server handler (inetd), 304–305
SNMP agent (snmpd), 312–317
SSH daemon (sshd), 317–322
TFTP daemon (tftpd), 310–311
network stacks, 188–189
Network Time Protocol (NTP) daemon (ntpd(8)), 44
configuring, 294
using, 296
network.conf file, 247, 248
MASTER_SITE_OVERRIDE variable, 249
networking
setup after install, 62–65
default gateway, 64
dynamic configuration, 64
Ethernet interfaces, 62–64
name service servers, 65
starting in single-user mode, 72
newaliases(8), 65, 263
newcomers, in OpenBSD community, xxxiv
newfs(1), 133
newfs(8), 134–135, 139, 145
news facility, 283
newsyslog(8), 269, 290
NFS (Network File System)
clients, 159–160
permitted, 158–159
multiple exports for one partition, 159
read-only mounts, 157
root directory setup, 455–456
setup, 154–155
and users, 157–158
NFS server, 155–159
nfsd(8) daemon, 155
nginx web server, 229
NKMEMPAGES value in kernel, 351
noatime mount option, in FFS, 137
noauto mount option, 128
in FFS, 138
nobody account, 103, 158
nodev mount option, 128
in FFS, 137

nodump file flag, 176
noexec mount option, in FFS, 137–138
nologin variable, 97
normal optimization for PF, 419
nosuid mount option, 128
NTFS filesystem, 151
NTFS partitions (Microsoft), 150
ntpd(8) (NTP [Network Time Protocol] daemon), 44, 60
  configuring, 294
  using, 296
ntpd_flags variable, 59
ntrw.exe program, 40
null modem cable, for serial console, 75–76
numerical sysctls, 346

O

OATH one-time passwords, 100
official CDs, 20, 40
  package files on, 228
  upgrading from, 371, 373–375
offset
  for disklabel partition, 52
  for MBR partition, 34
on keyword, for packet filtering interface matching, 405
one-to-one NAT, 429
open code, 2
open ports, testing for TCP, 198
Open Shortest Path First (OSPF), 203
Open Systems Interconnection (OSI) protocol stack, 184–187
  Applications layer, 186–187
  Datalink layer, 185, 187
  Network layer, 185–186, 187
  Physical layer, 184, 187
  Transport layer, 186, 187
OpenBGPD directory, 21
OpenBSD. *See also* desktop OpenBSD
  birth of, xxxiii
  build your own
    preparations for, 383–388
    reasons to, 382–383
  choosing install media, 22–23
  community, xxxiv
    information sources, 1–2
  customizing, 449–460
    diskless installation, 450–454
    installations, 458–460
    running diskless, 454–456
    upgrades, 460
    USB installation media, 457
    virtualization, 450
  FAQ, 8, 37
  information sources, 3–10
    mailing lists, 8–10
    man pages, 3–7
    manual, 4
  obtaining a copy of, 19–23
    boot media, 22
    Internet downloads, 20
    mirror site layout, 20–21
    official CDs, 20, 40
    release directories, 21–22
  problem-solving resources, 10–14
  security announcements, 172
  security flaws, 170
  source of problems, 2
  strengths, xxxv–xxxix
  support model, 2
  upgrade process, 371–373
  uses, xl
  versions, 368–371
  website, 7–8
    for hardware compatibility lists, 38
OpenBSD Ports website, 230
OpenBSD-specific functions, MIBs for, 313
OpenBSD Upgrade Guide, 371–373, 460
openfiles variable, 96
OpenNTPD daemon, 60, 294
OpenNTPD directory, 21
OpenOffice, 336
OpenSSH directory, 21
operating systems
  /usr partition for programs, 28
  installing multiple on one computer, 37
  multiple, 19
  source code for, installing, 66
Oracle VirtualBox, 450
orphaned devices, custom kernels and, 364
OSI. *See* Open Systems Interconnection (OSI) protocol stack
OSPF (Open Shortest Path First), 203
ospfd(8), 205
out keyword, for direction in packet filtering, 404–405
outgoing traffic, control of, 400
overlapping names, for man pages, 6
ownership, foreign filesystems and, 152

P

p command, 50
package respository, updating, 380–381